

Membedah Large Language Model (LLM)

Apa itu LLM dan bagaimana ia dibuat dari nol: data, tokenisasi, Transformer, backpropagation, pretraining, post-training, hingga inferensi, dengan bahasa teknis dan penjelasan awam, tanpa rumus

Dokumen dibuat 23 September 2026. Disusun dari riset berbagai sumber; rujukan tiap klaim tercantum sebagai catatan kaki di akhir dokumen. Data dan angka berlaku per tanggal sumbernya.

Daftar Isi

Pendahuluan

- Ruang lingkup dan batasan
- Cara membaca

Gambaran Besar

- Satu kalimat yang menjelaskan hampir semuanya
- Peta jalan: dari teks mentah sampai asisten
- Fakta kunci

Fondasi: Apa Itu "Model Bahasa"?

- Definisi: mesin probabilitas atas teks
- Mengapa "large"?
- Sejarah singkat: dari menghitung frekuensi ke Transformer

Jaringan Saraf dan Cara Belajarnya

- Neuron tiruan: kenop, penjumlahan, dan saklar
- Lapisan dan kedalaman
- Belajar = mengurangi kesalahan
- Gradient descent: menuruni bukit dalam kabut
- Backpropagation: rantai tanggung jawab dari belakang ke depan
- Optimizer: cara melangkah yang lebih cerdas
- Generalisasi, overfitting, dan mengapa data harus beragam
- Stabilitas pada jaringan sangat dalam: residual dan normalisasi

Data dan Tokenisasi: Bahan Baku LLM

- Dari mana datanya?
- Membersihkan internet
- Resep campuran data (data mix)
- Tokenisasi: memotong teks menjadi keping bernomor
- Embedding: memberi setiap token koordinat makna

Arsitektur Transformer: Mesin di Balik LLM

- Self-attention: setiap kata bertanya, setiap kata menawarkan
- Posisi: memberi tahu urutan kata
- Feed-forward network: tempat "fakta" banyak tersimpan
- Menumpuk blok: kedalaman dan residual stream
- Mixture of Experts (MoE): model besar yang hemat
- Berapa banyak parameter, dan di mana?

Pretraining: Membaca Internet dan Menebak Kata Berikutnya

Tujuannya sederhana: next-token prediction
Satu langkah pelatihan, dari awal sampai akhir
Hukum penskalaan (scaling laws)
Infrastruktur: pabrik komputasi
Hasilnya: base model

Post-training: Dari Pembaca Internet Menjadi Asisten

Tahap 1: Supervised Fine-Tuning (SFT)
Tahap 2: Belajar dari preferensi (RLHF, DPO, RLAIIF)
Tahap 3: Reinforcement learning untuk penalaran
Pelatihan keamanan, kepribadian, dan penggunaan alat
Distilasi: guru besar, murid kecil
Ringkasan post-training

Evaluasi: Bagaimana Tahu Modelnya Bagus?

Jenis-jenis evaluasi
Masalah-masalah evaluasi

Inferensi: Apa yang Terjadi Saat Anda Bertanya

Langkah demi langkah
Sampling: memilih dari distribusi peluang
KV cache: tidak menghitung ulang yang sudah lewat
Jendela konteks (context window)
Membuat inferensi lebih murah dan cepat
Memperluas kemampuan saat inferensi: RAG, alat, dan agen

Resep Lengkap: Membangun LLM dari Nol

Langkah 0: Tentukan tujuan dan anggaran
Langkah 1: Bangun korpus data
Langkah 2: Latih tokenizer
Langkah 3: Rancang arsitektur
Langkah 4: Siapkan infrastruktur
Langkah 5: Pretraining
Langkah 6: Post-training
Langkah 7: Evaluasi dan red-teaming
Langkah 8: Optimasi dan penerapan

Batasan dan Miskonsepsi

Halusinasi
Miskonsepsi umum

Batasan lain

Data & Angka Kunci

Skala model-model tonggak (hanya yang datanya dipublikasikan)

Tren komputasi, biaya, dan sumber daya

Data dan batasnya

Post-training: angka dari InstructGPT

Perdebatan & Perspektif

Apakah LLM "memahami"? Burung beo stokastik vs model dunia

Kemampuan emergen: nyata atau ilusi pengukuran?

Apakah penskalaan akan terus berhasil?

Apakah data akan habis? Data sintetis dan "keruntuhan model"

Model terbuka vs tertutup

Etika, hak cipta, dan dampak sosial

Kondisi Terkini & Prospek

Keadaan saat ini

Tren dan pertanyaan ke depan

Kronologi

Glosarium

Kesimpulan

Bacaan Lanjutan

Pendahuluan

Pada 30 November 2022, OpenAI membuka ChatGPT untuk umum. Dua bulan kemudian, menurut estimasi analisis UBS berbasis data Similarweb, layanan itu sudah dipakai sekitar 100 juta orang per bulan, laju adopsi tercepat yang pernah tercatat untuk aplikasi konsumen⁵⁹. Tiga setengah tahun kemudian, Stanford AI Index 2026 mencatat bahwa AI generatif telah mencapai tingkat adopsi 53% populasi dalam tiga tahun, lebih cepat daripada komputer pribadi maupun internet⁵⁸.

Di balik semua itu ada satu jenis program yang disebut **Large Language Model (LLM)**, "model bahasa berukuran besar". Kebanyakan orang memakainya setiap hari tanpa tahu apa yang sebenarnya terjadi di dalamnya. Ada yang membayangkan LLM sebagai mesin pencari yang pandai merangkai kata, ada yang mengira ia menyimpan "database jawaban", ada pula yang yakin ia sudah "berpikir seperti manusia". Ketiga gambaran itu keliru, masing-masing dengan cara yang berbeda.

Dokumen ini berusaha menjawab dua pertanyaan besar:

1. **LLM itu sebenarnya apa?** Apa yang dihitung, disimpan, dan dihasilkannya?
2. **Bagaimana LLM dibuat dari nol?** Mulai dari mengumpulkan teks internet, mengubahnya menjadi angka, merancang arsitektur jaringan saraf (Transformer), melatihnya dengan *backpropagation* pada ribuan GPU (*pretraining*), lalu "mendidiknya" menjadi asisten yang berguna dan aman (*post-training*: SFT, RLHF, DPO, dan *reinforcement learning* untuk penalaran), sampai ia melayani pengguna (*inferensi*).

I Ruang lingkup dan batasan

Sesuai permintaan, dokumen ini **tidak memakai rumus matematika**. Semua mekanisme dijelaskan dengan dua lapis bahasa:

- **Bahasa teknis**, dengan istilah yang dipakai para peneliti (*gradient*, *loss*, *attention*, *reward model*, dan sebagainya), supaya pembaca bisa melanjutkan ke literatur aslinya.
- **Bahasa mudah**, berupa analogi dan contoh sehari-hari, supaya intuisinya tertanam.

Dokumen ini juga **bukan tutorial pemrograman**. Tujuannya adalah pemahaman konseptual yang utuh, cukup dalam sehingga pembaca mengerti *mengapa* setiap tahap ada, apa yang terjadi kalau tahap itu dilewati, dan di mana letak batas-batasnya. Fokusnya adalah LLM berbasis teks. Model multimodal (yang juga memahami gambar, suara, video) disinggung seperlunya karena prinsip dasarnya sama.

SOAL ANGKA DAN NAMA MODEL

Dunia LLM bergerak sangat cepat. Angka-angka dalam dokumen ini selalu disertai tahun dan sumbernya. Banyak laboratorium besar sekarang tidak lagi membuka detail pelatihan model andalan mereka; Stanford AI Index 2026 mencatat skor rata-rata indeks transparansi model fondasi justru turun dari 58 menjadi 40⁵⁸. Karena itu, contoh-contoh teknis di sini sengaja banyak diambil dari model yang **laporannya terbuka** (GPT-3, Chinchilla, Llama 3, DeepSeek-V3/R1), bukan dari model tertutup yang detailnya hanya bisa ditebak.

I Cara membaca

Pembaca yang ingin gambaran cepat bisa membaca **Gambaran Besar**, lalu **Resep Lengkap: Membangun LLM dari Nol** dan **Kesimpulan**. Pembaca yang ingin memahami mesinnya sebaiknya membaca berurutan, karena setiap bagian dibangun di atas bagian sebelumnya: jaringan saraf dan backpropagation → data dan token → Transformer → pretraining → post-training → inferensi. Istilah-istilah teknis dijelaskan saat pertama kali muncul dan dirangkum di **Glosarium**.

Gambaran Besar

| Satu kalimat yang menjelaskan hampir semuanya

Kalau harus diringkas dalam satu kalimat: **LLM adalah jaringan saraf raksasa yang dilatih untuk menebak potongan kata berikutnya dalam sebuah teks, dan dari latihan menebak yang dilakukan triliunan kali itu, ia terpaksa mempelajari tata bahasa, fakta, gaya, dan pola penalaran yang tersimpan dalam bahasa manusia.**

Bayangkan fitur *autocomplete* di papan ketik ponsel yang menebak kata berikutnya saat Anda mengetik. LLM pada dasarnya adalah autocomplete, tetapi dengan tiga perbedaan skala yang mengubah segalanya:

- Ia membaca **jauh lebih banyak**: belasan triliun potongan kata, setara jutaan kali isi seluruh Wikipedia bahasa Indonesia.
- Ia punya "**otak**" **jauh lebih besar**: puluhan hingga ratusan miliar angka yang bisa disetel (disebut *parameter*).
- Ia melihat **konteks jauh lebih panjang**: bukan hanya dua-tiga kata sebelumnya, tetapi ribuan hingga ratusan ribu kata sekaligus.

Untuk menebak dengan baik kata berikutnya dalam kalimat "Tbu kota Jepang adalah ...", model harus "tahu" fakta geografi. Untuk menebak baris berikutnya dari sebuah kode program, ia harus "mengerti" logika pemrograman. Untuk menebak langkah berikutnya dari penyelesaian soal matematika, ia harus meniru pola penalaran. Inilah paradoks yang membuat LLM begitu menarik: tujuan latihannya sangat sederhana, tetapi untuk mencapainya dengan baik, model dipaksa membangun representasi internal yang kaya tentang dunia.

| Peta jalan: dari teks mentah sampai asisten

Pembuatan LLM modern bisa dibagi menjadi enam tahap besar. Andrej Karpathy, salah satu pendiri OpenAI, memakai pembagian serupa dalam kuliah populernya tahun 2025⁶⁰.

Tahap	Istilah teknis	Bahasa mudah	Hasil
1. Data	<i>Data collection, filtering, deduplication</i>	Mengumpulkan dan membersihkan "perpustakaan" raksasa dari internet, buku, kode	Korpus teks bersih, belasan triliun token
2. Tokenisasi	<i>Tokenization (BPE)</i>	Memotong teks menjadi keping-keping kecil dan memberi nomor	Deretan angka (ID token)
3. Arsitektur	<i>Transformer, self-attention</i>	Merancang "otak" kosong dengan miliaran kenop yang bisa disetel	Model dengan parameter acak
4. Pretraining	<i>Next-token prediction, backpropagation, gradient descent</i>	Menyuruh model menebak kata berikutnya triliunan kali dan mengoreksinya setiap kali salah	<i>Base model</i> : "pembaca internet" yang pandai melanjutkan teks
5. Post-training	<i>SFT, RLHF, DPO, RL with verifiable rewards</i>	"Menyekolahkan" model agar menjadi asisten: menjawab pertanyaan, sopan, jujur, bisa bernalar	<i>Instruct/chat model, reasoning model</i>
6. Inferensi	<i>Sampling, KV cache, quantization</i>	Menjalankan model untuk melayani pengguna, satu token demi satu token	Jawaban yang Anda baca di layar

Di sela-sela tahapan itu ada **evaluasi** (mengukur kemampuan dan keamanan) yang dilakukan terus-menerus.

ANALOGI KESELURUHAN: DARI PEMBACA RAKUS MENJADI PROFESIONAL

Pretraining itu seperti seorang anak yang dikurung di perpustakaan terbesar di dunia dan membaca semuanya tanpa bimbingan: novel, forum, jurnal, kode, resep, gosip. Ia jadi tahu banyak hal, tetapi tidak tahu cara bersikap: kalau ditanya, ia mungkin melanjutkan pertanyaan Anda dengan pertanyaan lain, karena di internet pertanyaan sering muncul berderet.

Post-training adalah sekolah profesi. Mula-mula ia diberi contoh-contoh jawaban teladan (SFT). Lalu ia diberi umpan balik tentang jawaban mana yang lebih baik (RLHF/DPO). Terakhir, ia berlatih mengerjakan soal-soal yang jawabannya bisa dicek sampai terampil bernalar (RL dengan hadiah terverifikasi). Pengetahuannya hampir seluruhnya berasal dari masa membaca di perpustakaan; sekolah profesi terutama membentuk **cara** ia memakai pengetahuan itu.

Fakta kunci

Aspek	Fakta	Sumber
Arsitektur dasar	Transformer, diperkenalkan 2017 dalam makalah "Attention Is All You Need"	Vaswani dkk., 2017 ¹
Algoritma belajar	Backpropagation + gradient descent (dipopulerkan 1986)	Rumelhart dkk., 1986 ²
Tujuan pretraining	Menebak token berikutnya (<i>next-token prediction</i>)	GPT-1 s.d. GPT-3 ¹⁴¹⁷
Skala data (contoh terbuka)	Llama 3 405B: ±15,6 triliun token; DeepSeek-V3: 14,8 triliun token	Meta 2024 ¹⁸ ; DeepSeek 2024 ²⁰
Skala parameter (contoh)	GPT-3: 175 miliar; Llama 3: 405 miliar; DeepSeek-V3: 671 miliar (37 miliar aktif per token)	171820
Komputasi (contoh)	Llama 3.1 405B: 30,84 juta jam-GPU H100	Model card Llama 3.1 ¹⁹
Biaya komputasi	Transformer asli (2017) ±US\$670; GPT-4 ±US\$78 juta; Gemini Ultra ±US\$192 juta (estimasi)	AI Index 2025 ⁵⁷
Pertumbuhan komputasi	Komputasi pelatihan model terdepan tumbuh ±4–5× per tahun (2010–2024)	Epoch AI ²⁸
Post-training klasik	SFT → reward model → RL (PPO), dipopulerkan InstructGPT	Ouyang dkk., 2022 ³³
Model penalaran	RL dengan hadiah terverifikasi memunculkan <i>chain-of-thought</i> panjang	DeepSeek-R1, Nature 2025 ³⁷

Fondasi: Apa Itu "Model Bahasa"?

I Definisi: mesin probabilitas atas teks

Secara teknis, **model bahasa** (*language model*) adalah sistem yang memberi **probabilitas** pada urutan kata. Diberi awal sebuah teks, ia menghasilkan daftar semua kemungkinan kelanjutannya beserta peluang masing-masing.

Misalnya, diberi teks "Saya minum kopi setiap ...", model yang baik akan memberi peluang tinggi pada "pagi", cukup tinggi pada "hari" dan "malam", dan peluang sangat kecil pada "kulkas". Model tidak "memilih" satu jawaban; ia menghasilkan **distribusi peluang** atas seluruh kosakata. Memilih satu kata dari distribusi itu adalah langkah terpisah yang disebut *sampling* (dibahas di bagian Inferensi).

Setelah satu kata dipilih, kata itu ditempelkan ke teks, lalu model diminta menebak lagi. Begitu terus, satu keping demi satu keping. Sifat ini disebut **autoregresif**: setiap keluaran menjadi bagian dari masukan langkah berikutnya. Seluruh esai panjang, kode program, atau puisi yang Anda terima dari chatbot dihasilkan dengan cara ini, satu keping pada satu waktu.

KATA ATAU TOKEN?

Sebenarnya LLM tidak bekerja dengan "kata", melainkan dengan **token**, yaitu potongan teks yang bisa berupa kata utuh, bagian kata, tanda baca, atau spasi. Kata "mempertanggungjawabkan" mungkin dipecah menjadi beberapa token. Di bagian ini kita pakai "kata" demi kemudahan; detail token dibahas di bagian Data dan Tokenisasi.

I Mengapa "large"?

Kata *large* merujuk terutama pada dua hal: **jumlah parameter** (angka-angka yang disetel selama pelatihan) dan **jumlah data** yang dipakai melatihnya. Tidak ada batas resmi kapan sebuah model bahasa disebut "large". Pada 2018, GPT-1 dengan ratusan juta parameter sudah dianggap besar; kini model dengan 7–8 miliar parameter sering disebut "kecil". Yang penting adalah temuan empiris bahwa **memperbesar skala secara konsisten menurunkan kesalahan prediksi** mengikuti pola yang bisa diprediksi (*scaling laws*), dan bahwa pada skala besar muncul kemampuan yang tidak tampak pada model kecil²³¹⁷.

I Sejarah singkat: dari menghitung frekuensi ke Transformer

Memahami sejarahnya membantu memahami *mengapa* arsitektur LLM seperti sekarang. Setiap generasi menambal kelemahan generasi sebelumnya.

Era statistik: n-gram (1980-an–2000-an)

Model bahasa awal sangat sederhana: hitung seberapa sering sebuah kata muncul setelah dua atau tiga kata tertentu dalam kumpulan teks besar. Ini disebut **model n-gram**. Kalau "minum kopi setiap" diikuti "pagi" 500 kali dan "malam" 200 kali, maka peluang "pagi" lebih besar.

Kelemahannya fatal: model n-gram hanya melihat beberapa kata terakhir, dan ia tidak tahu bahwa "kopi" dan "teh" itu mirip. Kalau frasa "minum teh setiap" belum pernah muncul di data, model tidak bisa meminjam pengetahuan dari "minum kopi setiap". Masalah ini disebut **kutukan dimensi** (*curse of dimensionality*): jumlah kombinasi kata tumbuh terlalu cepat untuk bisa dihitung semua⁴.

Kata sebagai titik dalam ruang: model bahasa neural (2003) dan word2vec (2013)

Pada 2003, Yoshua Bengio dan rekan-rekannya mengusulkan solusi yang menjadi benih seluruh LLM modern: wakili setiap kata sebagai **vektor**, yaitu deretan angka yang menunjukkan posisinya dalam "ruang makna" berdimensi banyak, lalu biarkan jaringan saraf mempelajari posisi itu sendiri⁴. Kata-kata yang dipakai dalam konteks serupa ("kopi", "teh", "susu") akan berdekatan, sehingga pengetahuan tentang satu kata bisa "menular" ke kata yang mirip.

Satu dekade kemudian, **word2vec** (Mikolov dkk., Google, 2013) menunjukkan bahwa representasi seperti itu bisa dilatih sangat efisien pada miliaran kata, dan bahwa arah-arrah dalam ruang itu menangkap hubungan makna. Contoh terkenalnya: vektor "raja" dikurangi "pria" ditambah "wanita" menghasilkan titik yang dekat dengan "ratu"⁵. Representasi vektor semacam ini disebut **embedding**, dan sampai hari ini merupakan lapisan pertama setiap LLM.

Membaca berurutan: RNN dan LSTM

Masalah berikutnya adalah konteks panjang. **Recurrent Neural Network (RNN)** membaca teks kata demi kata sambil membawa semacam "ingatan" yang diperbarui di setiap langkah. Variannya, **LSTM** (Hochreiter & Schmidhuber, 1997), menambahkan "gerbang" yang mengatur apa yang diingat dan dilupakan, sehingga bisa menjaga informasi lebih lama⁶. Pada 2014, arsitektur *sequence-to-sequence* berbasis LSTM mencapai terobosan dalam penerjemahan mesin⁷.

Namun RNN punya dua kelemahan struktural. Pertama, ingatannya adalah satu "botol" berukuran tetap: seluruh isi kalimat panjang harus dijejalkan ke sana, sehingga informasi awal cenderung memudar. Kedua, karena membaca berurutan, pelatihannya **tidak bisa diparalelkan**: kata ke-100 baru bisa diproses setelah kata ke-99 selesai. Ini membuat RNN sulit dilatih pada data raksasa dengan GPU yang justru unggul dalam komputasi paralel.

Perhatian (attention): 2014

Bahdanau, Cho, dan Bengio (2014) memperkenalkan mekanisme **attention** untuk penerjemahan: alih-alih mengandalkan satu botol ingatan, saat menghasilkan setiap kata terjemahan, model boleh "menoleh" ke semua kata di kalimat sumber dan menimbang mana yang paling relevan⁸. Ini seperti

penerjemah manusia yang terus melirik ke teks aslinya, bukan menghafal seluruh kalimat lalu menutup buku.

Transformer: "Attention Is All You Need" (2017)

Delapan peneliti Google kemudian mengambil langkah radikal: **buang RNN sama sekali, dan bangun model hanya dari attention** (ditambah lapisan-lapisan sederhana). Arsitektur ini dinamai **Transformer**¹. Karena setiap kata bisa langsung "melihat" semua kata lain tanpa harus menunggu giliran, seluruh kalimat bisa diproses **serentak**. Pelatihan menjadi jauh lebih cepat di GPU, dan jarak antarkata tidak lagi menjadi hambatan. Transformer asli mencetak rekor penerjemahan Inggris–Jerman (28,4 BLEU) dengan biaya pelatihan jauh lebih kecil daripada pendahulunya¹. Menurut estimasi AI Index, biaya komputasi melatih Transformer asli hanya sekitar US\$670⁵⁷.

Keluarga GPT dan BERT (2018–2020)

Transformer kemudian dipakai dengan dua cara:

- **BERT** (Google, 2018) memakai bagian *encoder* Transformer dan dilatih menebak kata yang disembunyikan di tengah kalimat. Hebat untuk memahami dan mengklasifikasi teks, tetapi tidak dirancang untuk menulis panjang¹⁵.
- **GPT** (*Generative Pre-trained Transformer*, OpenAI) memakai bagian *decoder* dan dilatih menebak kata berikutnya. GPT-1 (2018) menunjukkan bahwa pra-latih (*pre-training*) pada teks umum lalu penyesuaian (*fine-tuning*) pada tugas khusus sangat efektif¹⁴. GPT-2 (2019, 1,5 miliar parameter) menunjukkan bahwa model yang cukup besar bisa mengerjakan banyak tugas **tanpa dilatih khusus**, hanya dengan diberi teks pemicu yang tepat¹⁶. GPT-3 (2020, 175 miliar parameter) membuktikan kemampuan *few-shot learning*: diberi beberapa contoh di dalam prompt, model bisa mengerjakan tugas baru tanpa satu pun parameter diubah¹⁷.

Hampir semua LLM generatif modern (GPT, Claude, Gemini, Llama, DeepSeek, Qwen, Mistral) mewarisi resep GPT: **Transformer decoder-only yang dilatih menebak token berikutnya**, lalu dipoles dengan post-training.

ChatGPT dan era asisten (2022–sekarang)

Kemampuan GPT-3 sudah luar biasa, tetapi ia sulit dipakai orang awam: ia melanjutkan teks, bukan menjawab perintah. Terobosan yang membuat ChatGPT meledak bukanlah model yang jauh lebih besar, melainkan **post-training**: teknik InstructGPT (2022) yang memakai umpan balik manusia untuk mengajari model mengikuti instruksi. Hasilnya mengejutkan: model InstructGPT berukuran 1,3 miliar parameter lebih disukai penilai manusia daripada GPT-3 175 miliar yang belum dipoles³³. Pelajaran ini akan kita bahas panjang di bagian Post-training.

Jaringan Saraf dan Cara Belajarnya

Sebelum masuk ke Transformer, kita perlu memahami bahan bakunya: **jaringan saraf tiruan** (*artificial neural network*) dan algoritma yang membuatnya bisa belajar, **backpropagation**. Bagian ini adalah jantung dari pertanyaan "bagaimana mesin bisa belajar?"

I Neuron tiruan: kenop, penjumlahan, dan saklar

Satu **neuron tiruan** melakukan pekerjaan yang sangat sederhana:

1. Menerima beberapa angka sebagai masukan.
2. Mengalikan setiap masukan dengan sebuah **bobot** (*weight*), yang menunjukkan seberapa penting masukan itu.
3. Menjumlahkan semuanya, ditambah satu angka penyesuai yang disebut **bias**.
4. Melewatkan hasilnya ke sebuah **fungsi aktivasi**, semacam saklar lentur yang menentukan seberapa kuat neuron itu "menyala".

Fungsi aktivasi ini penting karena ia **tidak lurus** (*non-linear*). Tanpa ketidaklurusan itu, menumpuk seribu lapisan neuron sama saja dengan satu lapisan, karena gabungan operasi lurus tetaplah operasi lurus. Dengan fungsi non-linear, tumpukan lapisan bisa meniru pola yang sangat rumit.

Bobot dan bias inilah yang disebut **parameter**. Ketika orang berkata "model 70 miliar parameter", maksudnya ada 70 miliar kenop angka seperti ini yang nilainya disetel selama pelatihan. **Seluruh "pengetahuan" LLM tersimpan dalam nilai-nilai kenop tersebut**, bukan dalam bentuk kalimat atau tabel fakta.

ANALOGI: MIXER AUDIO RAKSASA

Bayangkan meja *mixer* di studio rekaman dengan puluhan miliar kenop. Suara masuk di satu sisi, keluar di sisi lain. Pada awalnya semua kenop diputar acak, sehingga keluarannya berisik tidak karuan. Pelatihan adalah proses memutar kenop-kenop itu sedikit demi sedikit, jutaan kali, sampai keluarannya terdengar benar. Tidak ada satu kenop pun yang "berisi" lagu tertentu; lagu itu muncul dari **kombinasi** semua kenop.

I Lapisan dan kedalaman

Neuron disusun dalam **lapisan** (*layer*). Keluaran satu lapisan menjadi masukan lapisan berikutnya. Jaringan dengan banyak lapisan disebut **deep neural network**, dan bidang yang mempelajarinya disebut **deep learning**.

Mengapa kedalaman penting? Karena setiap lapisan bisa membangun konsep di atas konsep dari lapisan sebelumnya. Pada jaringan pengenalan gambar, lapisan awal mendeteksi garis dan tepi, lapisan tengah mendeteksi bentuk (mata, roda), lapisan akhir mendeteksi objek utuh (wajah, mobil). Pada LLM, pola serupa terjadi: lapisan awal cenderung menangani ejaan dan tata bahasa lokal, lapisan tengah menangani makna dan hubungan, lapisan akhir menyiapkan tebakan kata berikutnya. Pembagian ini tidak serapi itu dalam kenyataan, tetapi kecenderungannya terlihat dalam riset interpretabilitas⁵⁰.

Dalam praktik, lapisan-lapisan ini dihitung sebagai **perkalian matriks**: tabel angka besar dikalikan dengan tabel angka besar lainnya. Inilah alasan GPU, yang awalnya dirancang untuk grafis game, menjadi mesin utama AI: GPU sangat cepat melakukan jutaan perkalian dan penjumlahan secara paralel.

I Belajar = mengurangi kesalahan

Jaringan saraf "belajar" dengan cara yang sangat mekanis. Untuk LLM, satu langkah belajar kira-kira seperti ini:

1. **Umpan maju (*forward pass*)**: berikan potongan teks, misalnya "Ibu kota Jepang adalah", ke model. Model menghitung, lapisan demi lapisan, lalu mengeluarkan distribusi peluang untuk token berikutnya.
2. **Hitung kesalahan (*loss*)**: bandingkan tebakan model dengan jawaban sebenarnya di data ("Tokyo"). Kalau model hanya memberi peluang 2% pada "Tokyo", kesalahannya besar. Kalau 90%, kesalahannya kecil. Ukuran yang dipakai untuk LLM disebut **cross-entropy loss**; intuisinya adalah "seberapa terkejut model melihat jawaban yang benar". Semakin terkejut, semakin besar hukumannya.
3. **Cari tahu siapa yang salah (*backpropagation*)**: hitung, untuk **setiap** dari miliaran parameter, ke arah mana dan seberapa kuat parameter itu harus diubah agar kesalahan mengecil.
4. **Perbaiki (*gradient descent*)**: geser setiap parameter sedikit ke arah yang mengurangi kesalahan.
5. Ulangi dengan potongan teks berikutnya. Jutaan kali.

Langkah 3 dan 4 adalah inti dari seluruh cerita, jadi mari kita bedah.

I Gradient descent: menuruni bukit dalam kabut

Bayangkan semua kemungkinan pengaturan kenop sebagai sebuah lanskap pegunungan raksasa. Setiap titik di lanskap mewakili satu pengaturan parameter, dan ketinggiannya adalah besar kesalahan (*loss*) model pada pengaturan itu. Tujuan pelatihan adalah menemukan lembah yang serendah mungkin.

Masalahnya, lanskap ini punya miliaran dimensi (satu dimensi per parameter), dan kita berdiri di tengah kabut tebal: kita tidak bisa melihat di mana lembah terdalam. Yang bisa kita rasakan hanyalah **kemiringan tanah di bawah kaki**. Maka strateginya: rasakan ke arah mana tanah paling menurun, ambil satu langkah kecil ke arah itu, rasakan lagi, melangkah lagi.

- **Gradien** (*gradient*) adalah "kemiringan tanah" itu: untuk setiap parameter, ia mengatakan "kalau kenop ini diputar sedikit ke kanan, kesalahan naik atau turun, dan seberapa cepat".
- **Gradient descent** adalah strategi melangkah menurun mengikuti gradien.
- **Learning rate** adalah panjang langkah. Terlalu panjang, kita bisa melompati lembah atau bahkan terlempar ke lereng seberang (pelatihan menjadi tidak stabil, loss meledak). Terlalu pendek, kita butuh waktu tak berujung untuk sampai.

Dalam praktik, gradien tidak dihitung dari seluruh data sekaligus (terlalu mahal), melainkan dari sekelompok kecil contoh yang diambil acak, disebut **batch** atau *mini-batch*. Karena itu metodenya disebut **stochastic gradient descent (SGD)**: arah langkahnya sedikit "berisik", tetapi rata-rata benar, dan justru keberisikannya itu sering membantu model tidak terjebak di cekungan kecil.

Backpropagation: rantai tanggung jawab dari belakang ke depan

Gradient descent butuh gradien untuk **setiap** parameter. Bagaimana menghitungnya secara efisien untuk ratusan miliar parameter? Di sinilah **backpropagation** (*backward propagation of errors*, perambatan balik kesalahan) masuk.

Masalah yang dipecahkan

Cara naif untuk mengetahui pengaruh satu kenop terhadap kesalahan adalah: putar kenop itu sedikit, jalankan ulang seluruh model, lihat apakah kesalahan naik atau turun. Untuk 100 miliar kenop, itu berarti menjalankan model 100 miliar kali **untuk satu langkah belajar saja**. Mustahil.

Backpropagation memecahkan masalah ini dengan cerdas: **semua gradien dihitung dengan biaya kira-kira hanya dua sampai tiga kali biaya satu kali umpan maju**. Inilah yang membuat pelatihan jaringan saraf raksasa mungkin dilakukan.

Cara kerjanya (tanpa rumus)

Kuncinya adalah **aturan rantai** (*chain rule*) dari kalkulus, yang intinya berkata: kalau A memengaruhi B, dan B memengaruhi C, maka pengaruh A terhadap C adalah pengaruh A terhadap B dikali pengaruh B terhadap C.

Jaringan saraf adalah rantai panjang operasi: masukan → lapisan 1 → lapisan 2 → ... → lapisan terakhir → loss. Backpropagation berjalan **mundur** di rantai itu:

1. Mulai dari ujung: seberapa besar loss berubah jika keluaran lapisan terakhir berubah? Ini mudah dihitung.

2. Mundur satu lapisan: dengan mengetahui "sinyal kesalahan" di keluaran lapisan terakhir, hitung seberapa besar setiap bobot di lapisan itu bertanggung jawab, **dan** seberapa besar setiap masukan lapisan itu (yang merupakan keluaran lapisan sebelumnya) bertanggung jawab.
3. Teruskan sinyal kesalahan itu ke lapisan sebelumnya, dan ulangi.
4. Sampai di lapisan pertama, setiap parameter sudah punya gradiennya.

Karena setiap lapisan memakai ulang hasil perhitungan lapisan sesudahnya, tidak ada kerja yang diulang. Selama umpan maju, nilai-nilai antara (*activations*) disimpan di memori justru supaya bisa dipakai lagi saat perambatan balik. Inilah sebabnya pelatihan memakan memori jauh lebih banyak daripada sekadar menjalankan model.

ANALOGI: MENYALAHKAN SECARA ADIL DI PABRIK

Bayangkan pabrik roti dengan 50 stasiun berjajar. Di ujung jalur, pemeriksa mutu menemukan roti terlalu asin. Ia tidak menyuruh semua stasiun mengulang dari awal; ia mengirim catatan mundur ke stasiun ke-50: "hasilmu terlalu asin sekian persen". Stasiun 50 mengecek: sebagian karena takaran garamnya sendiri (itu yang ia perbaiki), sebagian karena adonan dari stasiun 49 memang sudah asin. Ia lalu mengirim catatan yang **sudah disesuaikan** ke stasiun 49, dan seterusnya sampai stasiun pertama. Dalam satu kali perjalanan mundur, setiap stasiun tahu persis seberapa besar andilnya dan harus berubah ke arah mana. Itulah backpropagation.

Siapa penemunya?

Makalah Rumelhart, Hinton, dan Williams di *Nature* (1986) biasanya dianggap tonggak yang mempopulerkan backpropagation. Mereka menunjukkan bahwa dengan metode ini, unit-unit "tersembunyi" di tengah jaringan belajar merepresentasikan fitur-fitur penting dari tugas secara mandiri². Namun teknik matematis intinya, *reverse-mode automatic differentiation*, sudah dipublikasikan oleh Seppo Linnainmaa pada 1970, dan Paul Werbos mengusulkan penerapannya pada jaringan saraf pada 1970-an. Sejarah penemuannya masih sering diperdebatkan³. Geoffrey Hinton kemudian dianugerahi Nobel Fisika 2024 (bersama John Hopfield) atas kontribusi fondasional pada jaringan saraf tiruan.

Hari ini, tidak ada peneliti yang menulis backpropagation secara manual. *Library* seperti PyTorch dan JAX melakukan **diferensiasi otomatis**: programmer cukup menulis umpan maju, dan komputer menghitung semua gradien sendiri.

I Optimizer: cara melangkah yang lebih cerdas

Gradient descent polos punya kelemahan: semua parameter melangkah dengan aturan yang sama, padahal sebagian parameter butuh langkah besar dan sebagian lain butuh langkah halus. Maka dikembangkan **optimizer** yang lebih cerdas. Hampir semua LLM dilatih dengan **Adam** (Kingma & Ba, 2014) atau variannya **AdamW**¹⁰¹¹.

Dalam bahasa mudah, Adam menambahkan dua kebiasaan:

- **Momentum:** ia mengingat arah langkah-langkah sebelumnya, seperti bola yang menggelinding menuruni bukit dan tidak langsung berbelok karena satu kerikil. Ini meredam keberisikan gradien.
- **Langkah adaptif per parameter:** parameter yang gradiennya selalu besar dan liar diberi langkah lebih hati-hati; parameter yang gradiennya kecil dan jarang diberi langkah lebih berani.

AdamW menambahkan **weight decay** yang "dipisahkan": dorongan kecil agar semua bobot cenderung mengecil, semacam pajak atas kerumitan, sehingga model tidak bergantung berlebihan pada beberapa bobot ekstrem¹¹. Konsekuensi praktisnya: Adam menyimpan dua angka tambahan untuk **setiap** parameter. Untuk model 100 miliar parameter, itu berarti ratusan miliar angka tambahan yang harus ada di memori selama pelatihan.

Selain itu, learning rate hampir tidak pernah tetap. Resep umumnya adalah **warmup** (mulai sangat kecil, naik bertahap di ribuan langkah awal agar model yang masih acak tidak "kaget"), lalu **decay** (turun perlahan menjelang akhir, agar model bisa "mengendap" di dasar lembah).

I Generalisasi, overfitting, dan mengapa data harus beragam

Tujuan pelatihan bukan menghafal data, melainkan **generalisasi**: bekerja baik pada teks yang belum pernah dilihat. Model yang terlalu pas dengan data latih sampai menghafal keanehan-keanehannya disebut **overfitting**, seperti murid yang menghafal kunci jawaban tahun lalu tetapi gagal menghadapi soal baru.

Untuk mendeteksinya, sebagian data disisihkan sebagai **data validasi** yang tidak pernah dipakai untuk belajar. Selama pelatihan, peneliti memantau loss pada data validasi. Menariknya, LLM besar yang dilatih pada data sangat beragam biasanya **hanya melihat sebagian besar datanya sekali atau beberapa kali saja** (satu putaran penuh atas data disebut satu **epoch**), sehingga overfitting klasik lebih jarang menjadi masalah utama dibandingkan pada model kecil. Yang tetap terjadi adalah **hafalan** parsial: teks yang muncul berulang kali di internet (lisensi perangkat lunak, kutipan terkenal, soal ujian populer) bisa terekam hampir kata per kata. Itulah salah satu alasan **deduplikasi** data sangat penting (dibahas di bagian berikut).

I Stabilitas pada jaringan sangat dalam: residual dan normalisasi

Satu masalah klasik jaringan dalam adalah **vanishing/exploding gradient**: ketika sinyal kesalahan dirambatkan mundur melewati puluhan lapisan, ia bisa menyusut hampir nol (lapisan awal tidak belajar) atau membengkak tak terkendali. Dua penemuan penting menjinakkannya, dan keduanya dipakai di setiap Transformer:

- **Koneksi residual** (*residual/skip connection*, He dkk., 2015): setiap lapisan tidak mengganti masukannya, melainkan **menambahkan** koreksi di atasnya⁹. Ini menciptakan "jalan tol" tempat informasi dan gradien bisa mengalir lurus dari ujung ke ujung tanpa terkikis. Dalam LLM, jalur ini disebut **residual stream**, semacam papan tulis bersama yang dibaca dan ditulisi setiap lapisan secara bergantian.
- **Normalisasi lapisan** (*layer normalization*, dan varian ringannya *RMSNorm*): menyetel ulang skala angka di setiap lapisan agar tetap berada di rentang yang wajar, seperti pengatur volume otomatis yang mencegah suara terlalu pelan atau pecah.

Data dan Tokenisasi: Bahan Baku LLM

Ada pepatah lama di dunia komputer: *garbage in, garbage out*. Untuk LLM, pepatah itu berlaku dalam skala raksasa. Laporan teknis Llama 3 dari Meta secara eksplisit menyebut **data** sebagai salah satu dari tiga tuas utama peningkatan kualitas (dua lainnya adalah skala dan pengelolaan kerumitan), dan sebagian besar peningkatan Llama 3 dibanding Llama 2 berasal dari data yang lebih banyak dan lebih bersih, bukan dari arsitektur baru¹⁸.

I Dari mana datanya?

Korpus pretraining LLM modern biasanya merupakan campuran dari:

- **Halaman web**, terutama dari **Common Crawl**, organisasi nirlaba yang sejak 2008 merayapi web dan menerbitkan arsip berukuran petabyte secara berkala. Ini porsi terbesar.
- **Kode program** dari repositori publik. Kode ternyata tidak hanya membuat model pandai memprogram, tetapi juga dipercaya membantu penalaran terstruktur.
- **Buku, artikel ilmiah, ensiklopedia** (misalnya Wikipedia, arXiv), yang kualitas bahasanya tinggi dan padat pengetahuan.
- **Data matematika**, forum tanya-jawab, dan teks multibahasa.
- Semakin banyak, **data sintetis**: teks yang dihasilkan oleh model lain, lalu disaring (dibahas di bagian Perdebatan).

Untuk memberi gambaran skala: dataset terbuka **FineWeb** dari Hugging Face memproses 96 arsip Common Crawl (2013–2024) dan menghasilkan sekitar 15 triliun token teks bahasa Inggris yang sudah dibersihkan²⁵. Llama 3 dilatih pada sekitar 15,6 triliun token¹⁸; DeepSeek-V3 pada 14,8 triliun token²⁰.

SEBERAPA BESAR 15 TRILIUN TOKEN?

Satu token rata-rata kira-kira tiga perempat kata bahasa Inggris. Jadi 15 triliun token setara sekitar 11 triliun kata. Kalau seseorang membaca 250 kata per menit selama 8 jam sehari tanpa libur, ia butuh sekitar **250.000 tahun** untuk membaca semuanya. Karpathy menggambarkan FineWeb sebagai sekitar 44 terabyte teks, cukup muat di satu *hard disk* besar, karena teks sangat hemat ruang dibanding video⁶⁰.

I Membersihkan internet

Web mentah sebagian besar adalah sampah dari sudut pandang pelatihan: menu navigasi, iklan, halaman duplikat, spam, teks hasil mesin, konten berbahaya. *Pipeline* pembersihan umumnya

meliputi tahap-tahap berikut²⁵¹⁸:

1. **Ekstraksi teks:** membuang kode HTML, menu, dan *boilerplate*, lalu mengambil isi utama halaman.
2. **Penyaringan URL:** memblokir domain spam, situs dewasa, dan sumber berbahaya.
3. **Identifikasi bahasa:** mengelompokkan teks per bahasa. Bahasa dengan data sedikit (termasuk bahasa daerah) mendapat porsi kecil, dan ini salah satu sebab kemampuan LLM tidak merata antarbahasa.
4. **Filter kualitas heuristik:** aturan sederhana seperti membuang halaman yang terlalu pendek, terlalu banyak simbol, terlalu banyak baris berulang, atau rasio kata aneh.
5. **Deduplikasi:** menghapus dokumen yang identik atau hampir identik. Teknik seperti **MinHash** bisa mendeteksi kemiripan dokumen dalam skala miliaran. Tanpa deduplikasi, model membuang komputasi mempelajari hal yang sama berkali-kali dan cenderung menghafal teks yang berulang.
6. **Penghapusan data pribadi (PII):** menyamarkan alamat email, nomor telepon, dan sejenisnya.
7. **Filter kualitas berbasis model:** tren terbaru adalah memakai LLM untuk menilai halaman. Contohnya **FineWeb-Edu**: model Llama 3 70B diminta menilai "nilai edukatif" ratusan ribu halaman, penilaian itu dipakai melatih pengklasifikasi kecil, lalu pengklasifikasi itu menyaring seluruh korpus. Hasilnya, subset 1,3 triliun token yang menghasilkan model lebih baik pada tolok ukur pengetahuan dan penalaran dibanding korpus penuh yang jauh lebih besar²⁵.
8. **Dekontaminasi:** membuang teks yang mengandung soal-soal tolok ukur (benchmark) agar hasil evaluasi tidak "bocor".

I Resep campuran data (*data mix*)

Setelah bersih, data tidak dicampur begitu saja. Tim pelatih menentukan **proporsi**: berapa persen web umum, kode, matematika, multibahasa, dan seterusnya. Proporsi ini sangat memengaruhi watak model dan termasuk rahasia dapur paling dijaga. Laporan Llama 3 menyebut campuran akhirnya kira-kira 50% pengetahuan umum, 25% matematika dan penalaran, 17% kode, dan 8% multibahasa, yang ditentukan lewat eksperimen pada model-model kecil¹⁸.

Praktik umum lain adalah **annealing**: di bagian akhir pretraining, porsi data berkualitas sangat tinggi (misalnya matematika dan kode pilihan) dinaikkan sementara learning rate diturunkan, seperti "menutup semester dengan materi terbaik"¹⁸.

I Tokenisasi: memotong teks menjadi keping bernomor

Jaringan saraf hanya bisa mengolah angka. Maka teks harus diubah menjadi deretan angka. Langkah ini disebut **tokenisasi**, dan dilakukan oleh program terpisah bernama **tokenizer** yang disiapkan sebelum pretraining dan tidak berubah sesudahnya.

Mengapa tidak per huruf atau per kata?

- **Per huruf:** kosakatanya kecil, tetapi urutannya menjadi sangat panjang. Kalimat 100 kata menjadi ratusan langkah, dan model harus bekerja keras sekadar merangkai huruf menjadi kata.
- **Per kata:** urutan pendek, tetapi kosakatanya meledak (setiap bentuk kata, nama, salah ketik, istilah baru butuh nomor sendiri), dan kata yang tidak ada di kamus tidak bisa diproses sama sekali.

Jalan tengahnya adalah **subword**: keping-keping di antara huruf dan kata. Kata umum menjadi satu token utuh; kata langka dipecah menjadi beberapa keping yang sering muncul.

Byte Pair Encoding (BPE)

Algoritma yang paling luas dipakai adalah **Byte Pair Encoding (BPE)**, yang diadaptasi untuk bahasa oleh Sennrich dkk. (2016) dari algoritma kompresi data¹². Cara kerjanya sederhana:

1. Mulai dengan kosakata berisi semua karakter (atau semua 256 nilai *byte*, sehingga teks apa pun, termasuk emoji dan aksara non-Latin, pasti bisa direpresentasikan).
2. Hitung pasangan simbol bersebelahan yang paling sering muncul di korpus, misalnya "a" + "n".
3. Gabungkan pasangan itu menjadi simbol baru "an", tambahkan ke kosakata.
4. Ulangi puluhan ribu kali: "an" + "g" → "ang", "m" + "e" → "me", "me" + "ng" → "meng", dan seterusnya.

Proses berhenti ketika kosakata mencapai ukuran yang diinginkan. Ukuran kosakata LLM modern umumnya puluhan ribu hingga lebih dari 100 ribu token; Llama 3 memakai 128 ribu token¹⁸.

Konsekuensi tokenisasi yang sering tidak disadari

Tokenisasi menjelaskan beberapa keanehan LLM:

- **Kesulitan mengeja dan menghitung huruf.** Model melihat "stroberi" sebagai satu atau dua token, bukan delapan huruf. Pertanyaan "ada berapa huruf r di kata stroberi?" menuntut model mengetahui ejaan dari dalam token, sesuatu yang tidak ia "lihat" secara langsung.
- **Bahasa yang "mahal".** Karena tokenizer dilatih pada korpus yang didominasi bahasa Inggris, kata-kata bahasa Inggris cenderung jadi satu token, sedangkan kata-kata bahasa Indonesia atau bahasa lain lebih sering dipecah. Artinya teks yang sama panjang memakan lebih banyak token, sehingga lebih mahal (API dibayar per token) dan lebih cepat memenuhi batas konteks.
- **Angka dan aritmetika.** Angka panjang bisa dipecah secara tidak konsisten (misalnya "12345" menjadi "123" + "45"), sehingga aritmetika menjadi lebih sulit. Banyak tokenizer modern memotong angka per digit atau per tiga digit agar lebih teratur.

I Embedding: memberi setiap token koordinat makna

Setelah tokenisasi, setiap token hanyalah nomor, misalnya "kopi" = 48.213. Nomor itu tidak bermakna; 48.213 tidak "dekat" dengan 48.214. Maka langkah pertama di dalam model adalah **tabel embedding**: tabel raksasa yang memetakan setiap nomor token ke sebuah vektor, deretan ribuan angka (misalnya 4.096 atau 16.384 angka, tergantung ukuran model).

Vektor-vektor ini adalah parameter yang ikut dilatih. Pada awalnya acak; setelah pelatihan, token-token yang bermakna mirip memiliki vektor yang berdekatan, persis seperti ide Bengio (2003) dan word2vec (2013)⁴⁵. Bedanya, dalam Transformer, vektor ini hanyalah titik awal: lapisan-lapisan berikutnya akan terus memperkaya vektor itu dengan konteks, sehingga "bisa" dalam "bisa ular" dan "bisa" dalam "saya bisa" akhirnya punya representasi yang sangat berbeda.

Arsitektur Transformer: Mesin di Balik LLM

Sekarang kita masuk ke "otak" LLM. Hampir semua LLM generatif memakai **Transformer decoder-only**. Strukturnya, dari bawah ke atas:

1. **Tokenizer** mengubah teks menjadi deretan ID token.
2. **Embedding** mengubah setiap ID menjadi vektor.
3. **Tumpukan blok Transformer** (puluhan hingga lebih dari seratus blok yang identik strukturnya). Setiap blok berisi:
 - **Self-attention**: token-token "saling bertukar informasi".
 - **Feed-forward network (MLP)**: setiap token "berpikir sendiri" mengolah informasi yang terkumpul.
 - Di sekeliling keduanya: **normalisasi** dan **koneksi residual**.
4. **Lapisan keluaran (unembedding)** mengubah vektor token terakhir menjadi skor untuk setiap token di kosakata.
5. **Softmax** mengubah skor menjadi distribusi peluang.

Mari kita bedah bagian-bagian terpentingnya.

| Self-attention: setiap kata bertanya, setiap kata menawarkan

Masalah yang dipecahkan attention adalah **konteks**. Arti sebuah kata bergantung pada kata-kata lain. Dalam kalimat "Andi menaruh gelas di atas meja karena **ia** haus", kata "ia" merujuk pada Andi. Dalam "Andi menaruh gelas di atas meja karena **ia** licin", "ia" lebih masuk akal merujuk pada meja. Model harus bisa, untuk setiap kata, mencari kata-kata lain yang relevan dan menyerap informasinya.

Self-attention melakukannya dengan memberi setiap token tiga vektor turunan, yang dihasilkan dengan mengalikan vektor token dengan tiga matriks parameter berbeda:

- **Query (Q)**: "apa yang sedang saya cari?"
- **Key (K)**: "apa yang saya tawarkan / siapa saya?"
- **Value (V)**: "informasi apa yang akan saya berikan jika dipilih?"

Lalu untuk setiap token:

1. **Query**-nya dibandingkan dengan **Key** semua token lain (dengan operasi perkalian titik yang mengukur kecocokan).
2. Skor kecocokan diubah menjadi bobot perhatian yang jumlahnya 100% (lewat softmax).
3. Token itu mengambil **rata-rata tertimbang** dari **Value** semua token, sesuai bobot perhatian.

Hasilnya, vektor "ia" diperkaya dengan informasi dari "Andi" (atau "meja"), tergantung konteks.

ANALOGI: PESTA DAN PAPAN NAMA

Bayangkan setiap kata adalah tamu di sebuah pesta. Setiap tamu memakai **papan nama** yang menjelaskan dirinya (Key), membawa **pertanyaan** di kepalanya (Query), dan menyimpan **cerita** yang siap dibagikan (Value). Kata "ia" berkeliling dengan pertanyaan "siapa pelaku di kalimat ini yang bisa merasa haus?". Ia membaca papan nama semua tamu; papan "Andi: orang, pelaku" sangat cocok, "meja: benda" tidak. Maka "ia" mendengarkan cerita Andi dengan saksama dan cerita tamu lain sambil lalu. Setelah pesta, "ia" pulang dengan pemahaman baru tentang dirinya.

Multi-head attention

Satu jenis pertanyaan tidak cukup. Sebuah kata mungkin perlu tahu sekaligus: siapa subjeknya, kata kerja apa yang terkait, apakah kalimatnya negatif, bahasa apa yang dipakai, dan seterusnya. Maka setiap lapisan attention punya **banyak kepala (heads)**, masing-masing dengan matriks Q, K, V sendiri, yang berjalan paralel. Transformer asli memakai 8 kepala¹; model besar memakai puluhan hingga lebih dari seratus. Riset interpretabilitas menemukan bahwa kepala-kepala berbeda memang cenderung berspesialisasi, misalnya ada yang melacak hubungan tata bahasa, ada yang menyalin pola dari bagian teks sebelumnya.

Causal mask: tidak boleh mengintip masa depan

Dalam LLM, setiap token hanya boleh memperhatikan token **sebelumnya**, tidak pernah sesudahnya. Ini dipaksakan dengan **causal mask**, yang menutup semua posisi di masa depan. Alasannya: model dilatih menebak token berikutnya; kalau boleh mengintip, ia akan "menyontek" dan tidak belajar apa-apa.

Keuntungan besarnya: dalam satu kali umpan maju pada satu dokumen sepanjang 8.000 token, model sebenarnya mengerjakan **8.000 soal tebak-kata sekaligus** (tebak token ke-2 dari token ke-1, tebak ke-3 dari ke-1 dan ke-2, dan seterusnya), semuanya paralel. Inilah salah satu rahasia efisiensi pretraining Transformer dibanding RNN.

Harga attention: kuadratik

Karena setiap token membandingkan diri dengan semua token sebelumnya, biaya attention tumbuh secara **kuadratik** terhadap panjang teks: dua kali lebih panjang berarti sekitar empat kali lebih mahal. Ini alasan utama panjang konteks sempat menjadi hambatan besar. Banyak inovasi rekayasa berusaha mengatasinya, misalnya **FlashAttention** (Dao dkk., 2022), yang menghitung attention yang **persis sama** tetapi menata ulang urutan baca-tulis memori GPU sehingga jauh lebih cepat dan hemat memori³², serta **Grouped-Query Attention (GQA)** yang membuat beberapa kepala query berbagi key dan value sehingga memori saat inferensi berkurang drastis²².

I Posisi: memberi tahu urutan kata

Attention sendiri tidak peduli urutan; ia memperlakukan token seperti sekantong kata. Padahal "anjing menggigit orang" dan "orang menggigit anjing" jelas berbeda. Maka informasi **posisi** harus disuntikkan. Transformer asli menambahkan pola gelombang sinus ke setiap embedding¹. Kebanyakan LLM modern memakai **RoPE** (*Rotary Position Embedding*, Su dkk., 2021), yang "memutar" vektor query dan key dengan sudut yang bergantung pada posisinya, sehingga kecocokan antar dua token secara alami bergantung pada **jarak relatif** keduanya¹³. Salah satu keuntungan RoPE adalah memudahkan perpanjangan konteks setelah pelatihan.

I Feed-forward network: tempat "fakta" banyak tersimpan

Setelah attention mengumpulkan informasi dari token lain, setiap token melewati **feed-forward network (FFN)** atau **MLP**: dua lapisan neuron biasa dengan fungsi aktivasi di tengahnya, diterapkan pada setiap token secara terpisah. Lapisan tengahnya biasanya sekitar tiga sampai empat kali lebih lebar daripada vektor token, dan bagian inilah yang menyumbang sekitar dua pertiga parameter model.

Jika attention adalah "rapat koordinasi" antarkata, FFN adalah "waktu berpikir pribadi" setiap kata. Riset interpretabilitas menunjukkan bahwa banyak asosiasi faktual (misalnya "Menara Eiffel → Paris") tampaknya tersimpan terutama di lapisan-lapisan FFN, meskipun penyimpanan ini tersebar dan tidak serapi entri kamus.

I Menumpuk blok: kedalaman dan residual stream

Satu blok (attention + FFN) diulang berkali-kali. GPT-3 175B memakai 96 blok¹⁷; Llama 3 405B memakai 126 blok¹⁸. Karena koneksi residual, setiap blok tidak menulis ulang vektor token, melainkan **menambahkan** sesuatu ke dalamnya. Vektor setiap token berjalan melewati semua blok seperti dokumen yang diedarkan ke banyak meja, dan setiap meja menambahkan catatan.

Di ujung tumpukan, vektor token **terakhir** dikalikan dengan matriks *unembedding*, menghasilkan satu skor (*logit*) untuk setiap token di kosakata, misalnya 128.000 skor. **Softmax** mengubah skor-skor itu menjadi peluang yang jumlahnya 100%. Itulah tebakan model untuk token berikutnya.

I Mixture of Experts (MoE): model besar yang hemat

Model "padat" (*dense*) menjalankan **semua** parameternya untuk setiap token. Semakin besar model, semakin mahal setiap token. **Mixture of Experts (MoE)** mengakali ini: lapisan FFN diganti dengan banyak FFN kecil yang disebut **expert**, ditambah sebuah **router** yang, untuk setiap token, memilih hanya beberapa expert untuk dijalankan²¹.

- **Mixtral 8x7B** (2024) punya 8 expert per lapisan dan memilih 2 untuk setiap token: total 46,7 miliar parameter, tetapi hanya sekitar 13 miliar yang aktif per token²¹.
- **DeepSeek-V3** (2024) punya 671 miliar parameter total, tetapi hanya 37 miliar aktif per token²⁰.

Analoginya: rumah sakit besar dengan ratusan dokter spesialis. Pasien tidak diperiksa oleh semua dokter; petugas triase (router) mengirimnya ke dua atau tiga spesialis yang relevan. Rumah sakitnya "berpengetahuan" sangat luas, tetapi biaya per pasien tetap terkendali. MoE kini menjadi pilihan umum untuk model-model besar karena memberikan kapasitas pengetahuan besar dengan biaya komputasi per token yang lebih kecil. Tantangannya adalah menjaga agar beban terbagi merata (tidak semua token berebut ke expert yang sama) dan kebutuhan memori tetap besar karena semua expert harus tersimpan.

I Berapa banyak parameter, dan di mana?

Komponen	Fungsi	Porsi parameter (kira-kira, model dense)
Embedding & unembedding	Menerjemahkan token ↔ vektor	Kecil pada model besar (beberapa persen)
Attention (Q, K, V, output)	Pertukaran informasi antartoken	± sepertiga
Feed-forward / MLP	Pengolahan per token, banyak asosiasi tersimpan	± dua pertiga
Normalisasi	Menjaga skala angka	Sangat kecil

Angka ini perkiraan umum dan bervariasi antar-desain; pada model MoE, porsi FFN (para expert) jauh lebih besar lagi.

Pretraining: Membaca Internet dan Menebak Kata Berikutnya

Pretraining adalah tahap paling mahal, paling lama, dan paling menentukan. Di sinilah model memperoleh hampir seluruh pengetahuan dan kemampuan dasarnya. Karpathy menyebut hasil tahap ini sebagai semacam "kompresi internet yang *lossy*" ke dalam parameter model⁶⁰.

I Tujuannya sederhana: next-token prediction

Tujuan pretraining adalah **menebak token berikutnya** di seluruh korpus. Tidak ada label buatan manusia, tidak ada "jawaban benar" yang disiapkan khusus; teks itu sendiri adalah soal sekaligus kunci jawabannya. Karena itu pendekatan ini disebut **self-supervised learning**. Inilah yang memungkinkan pelatihan pada triliunan token: tidak butuh anotator manusia sama sekali.

I Satu langkah pelatihan, dari awal sampai akhir

Menggabungkan semua yang sudah kita bahas, satu **langkah pelatihan** (*training step*) berjalan seperti ini:

1. **Ambil batch**: ambil secara acak ribuan potongan teks dari korpus, masing-masing sepanjang panjang konteks (misalnya 8.192 token). Satu batch model besar bisa berisi belasan juta token.
2. **Umpan maju**: alirkan semua token melewati embedding dan seluruh blok Transformer. Untuk **setiap posisi** di setiap potongan, model mengeluarkan distribusi peluang token berikutnya.
3. **Hitung loss**: bandingkan setiap tebakan dengan token yang sebenarnya muncul; rata-ratakan tingkat "keterkejutan" (cross-entropy) di seluruh posisi.
4. **Backpropagation**: rambatkan kesalahan mundur melewati semua blok, sehingga setiap parameter (termasuk tabel embedding) mendapatkan gradiennya.
5. **Pembaruan optimizer**: AdamW menggeser setiap parameter sedikit, dengan learning rate sesuai jadwal.
6. **Ulangi**. Sebuah pretraining besar terdiri atas ratusan ribu hingga jutaan langkah seperti ini.

Di awal, loss sangat tinggi; model menebak hampir secara acak dan keluarannya adalah campuran token tak bermakna. Setelah ratusan langkah, ia belajar token-token yang sering muncul. Setelah ribuan langkah, ia mulai merangkai kata yang benar ejaannya. Kemudian tata bahasa, lalu koherensi paragraf, lalu fakta, gaya, dan pola penalaran. Grafik loss yang turun perlahan selama berminggu-minggu adalah "detak jantung" yang dipantau terus oleh tim pelatih.

MENGAPA MENEBAK KATA MEMBUAT MODEL 'PINTAR'?

Pikirkan teks seperti: "Setelah menambahkan 27 dan 58, hasilnya adalah ...". Untuk menurunkan loss pada ribuan contoh semacam ini, cara paling efisien bagi model bukan menghafal setiap pasangan angka, melainkan membangun mekanisme internal yang meniru penjumlahan. Demikian pula untuk menebak kalimat berikutnya dalam cerita detektif, akan sangat membantu jika model melacak siapa melakukan apa. Tekanan untuk memprediksi dengan tepat pada data yang sangat beragam mendorong model membentuk representasi yang merangkum struktur dunia di balik teks. Seberapa jauh ini bisa disebut "pemahaman" adalah perdebatan terbuka (lihat bagian Perdebatan).

I Hukum penskalaan (*scaling laws*)

Salah satu penemuan paling berpengaruh dalam sejarah LLM adalah bahwa kinerja model **bisa diprediksi** dari skalanya.

Kaplan dkk. (2020)

Tim OpenAI yang dipimpin Jared Kaplan menemukan bahwa loss model bahasa turun mengikuti **hukum pangkat** (*power law*) yang mulus terhadap tiga faktor: jumlah parameter, jumlah data, dan jumlah komputasi, dengan tren yang konsisten di rentang lebih dari tujuh orde besaran²³. Detail arsitektur seperti lebar atau kedalaman jaringan ternyata jauh kurang penting dalam rentang yang wajar. Artinya, sebelum membakar ratusan juta dolar untuk model besar, tim bisa melatih puluhan model kecil, menarik garis tren, dan memprediksi hasil model besar dengan cukup akurat. Ini mengubah pengembangan LLM dari seni menjadi sesuatu yang lebih mirip teknik.

Chinchilla (2022): data sama pentingnya dengan ukuran

Kaplan menyarankan bahwa dengan anggaran komputasi tertentu, lebih baik memperbesar model dan relatif menghemat data. Dua tahun kemudian, Hoffmann dkk. dari DeepMind mengoreksinya: untuk hasil optimal, **ukuran model dan jumlah token harus diperbesar dengan laju yang kira-kira sama**. Aturan praktisnya sering diringkas sebagai sekitar **20 token per parameter**²⁴. Mereka membuktikannya dengan melatih **Chinchilla** (70 miliar parameter, 1,4 triliun token) dengan anggaran komputasi yang sama seperti **Gopher** (280 miliar parameter, sekitar 300 miliar token). Chinchilla yang empat kali lebih kecil mengalahkan Gopher, GPT-3, dan model-model yang lebih besar lainnya di banyak tugas²⁴. GPT-3, dengan sekitar 1,7 token per parameter, ternyata "kurang makan data".

Setelah Chinchilla: sengaja "overtraining"

Resep Chinchilla mengoptimalkan **biaya pelatihan**. Tetapi model yang sudah jadi akan dijalankan miliaran kali untuk melayani pengguna, dan model kecil jauh lebih murah dijalankan. Maka praktik

sekarang adalah melatih model yang relatif kecil dengan data **jauh melebihi** rasio Chinchilla. Model Llama 3 8B, misalnya, dilatih dengan lebih dari 15 triliun token, ratusan kali lebih banyak dari 20 token per parameter¹⁸. Hasilnya model kecil yang sangat kuat, dengan harga pelatihan sedikit lebih mahal dari "optimal".

I Infrastruktur: pabrik komputasi

Pretraining model terdepan adalah salah satu proyek komputasi terbesar yang pernah dilakukan manusia.

Skala perangkat keras

Llama 3 405B dilatih memakai hingga sekitar 16.000 GPU NVIDIA H100 secara bersamaan¹⁸, dengan total 30,84 juta jam-GPU untuk model 405B saja, dan 39,3 juta jam-GPU untuk seluruh keluarga Llama 3.1¹⁹. DeepSeek-V3 melaporkan kebutuhan 2,788 juta jam-GPU H800 untuk pelatihan penuhnya, atau sekitar 180 ribu jam-GPU per triliun token, setara 3,7 hari pada kluster 2.048 GPU²⁰. Menurut Epoch AI, komputasi untuk melatih model-model terdepan tumbuh sekitar 4–5 kali lipat per tahun dari 2010 hingga 2024, terutama karena peningkatan belanja, bukan hanya perangkat keras yang lebih baik²⁸.

Paralelisme: membagi model ke ribuan GPU

Model 400 miliar parameter, beserta gradien dan status optimizer-nya, membutuhkan beberapa terabyte memori, jauh melebihi kapasitas satu GPU (puluhan hingga ratusan gigabyte). Maka pekerjaan dibagi dengan beberapa strategi sekaligus²⁹³⁰:

Strategi	Apa yang dibagi	Analogi
Data parallelism	Setiap kelompok GPU memegang salinan model, mengerjakan batch berbeda, lalu gradiennya dirata-ratakan	Banyak tim mengerjakan soal berbeda dengan buku yang sama, lalu menyatukan catatan koreksi
Tensor parallelism	Setiap perkalian matriks besar dipotong ke beberapa GPU	Satu halaman buku disobek menjadi beberapa bagian dan dibaca bersamaan
Pipeline parallelism	Lapisan-lapisan dibagi: GPU grup 1 memegang blok 1–30, grup 2 blok 31–60, dst.	Jalur perakitan pabrik
ZeRO / FSDP	Status optimizer, gradien, dan parameter dipecah (<i>sharded</i>) di antara GPU data-parallel	Tidak semua tim menyimpan seluruh buku; masing-masing menyimpan beberapa bab dan meminjamkannya saat perlu
Expert parallelism (MoE)	Expert-expert berbeda ditempatkan di GPU berbeda	Dokter spesialis di gedung berbeda

Kombinasi strategi ini sering disebut **paralelisme 3D atau 4D**. Mendesainnya agar ribuan GPU tidak saling menunggu adalah salah satu keahlian rekayasa paling langka di industri.

Presisi campuran

Angka di dalam komputer bisa disimpan dengan presisi tinggi (32 bit) atau rendah (16 atau 8 bit). Presisi rendah lebih cepat dan hemat memori, tetapi rawan galat pembulatan. **Mixed precision training** memakai presisi rendah untuk sebagian besar perhitungan sambil menyimpan salinan "induk" bobot dalam presisi tinggi³¹. Format **BF16** kini standar, dan DeepSeek-V3 bahkan melaporkan pelatihan skala besar dengan format **FP8** untuk banyak operasinya²⁰.

Hal-hal yang bisa (dan pasti) salah

Pada skala ribuan GPU selama berminggu-minggu, kegagalan adalah hal rutin. Laporan Llama 3 mencatat 466 gangguan kerja selama periode pretraining 54 hari, sebagian besar akibat masalah perangkat keras seperti GPU atau memori yang rusak¹⁸. Karena itu pelatihan dirancang dengan:

- **Checkpoint** berkala: seluruh keadaan model disimpan secara rutin, sehingga saat ada kegagalan, pelatihan bisa dilanjutkan dari titik terakhir.
- **Pemantauan loss spike**: kadang loss tiba-tiba melonjak (akibat data buruk atau ketidakstabilan numerik). Tim bisa mundur ke checkpoint sebelumnya, melewati batch bermasalah, atau menurunkan learning rate.

Biaya

Biaya pretraining sulit diketahui karena laboratorium jarang mengungkapkannya. AI Index 2025 mengestimasi biaya komputasi pelatihan GPT-4 sekitar US\$78 juta dan Gemini Ultra sekitar US\$192 juta⁵⁷. DeepSeek menghitung bahwa, dengan asumsi harga sewa US\$2 per jam-GPU, pelatihan akhir DeepSeek-V3 hanya bernilai sekitar US\$5,6 juta, tetapi mereka sendiri menegaskan angka itu **tidak mencakup** biaya riset, eksperimen sebelumnya, serta pembelian dan pengelolaan kluster²⁰. Angka ini sering dikutip keliru sebagai "biaya membuat DeepSeek".

| Hasilnya: base model

Pretraining menghasilkan **base model** (disebut juga *foundation model* atau *pretrained model*). Sifat-sifatnya:

- **Berpengetahuan luas:** ia mengenal banyak fakta, bahasa, gaya, kode.
- **Pandai melanjutkan teks:** beri awal artikel berita, ia menulis sisanya dengan gaya berita. Beri awal kode, ia melanjutkan kodenya.
- **Tidak "tahu" bahwa ia asisten.** Tanyakan "Apa ibu kota Jepang?" dan ia mungkin melanjutkan dengan "Apa ibu kota Korea? Apa ibu kota Cina?", karena di internet, pertanyaan kuis sering muncul berderet.
- **Meniru apa saja,** termasuk hal buruk yang ada di internet: kebencian, informasi salah, gaya kasar.
- **Bisa dipandu dengan contoh.** Dengan *few-shot prompting*, yaitu menaruh beberapa contoh tanya-jawab di awal teks, base model bisa diarahkan untuk menjawab, karena kini kelanjutan yang paling mungkin adalah jawaban¹⁷.

Karpathy menyebut base model sebagai "simulator dokumen internet": ia bisa memerankan penulis dokumen apa pun, tetapi tidak punya kepribadian atau tujuan sendiri⁶⁰. Tugas post-training adalah memilih dan menguatkan satu peran tertentu dari sekian banyak kemungkinan itu: **asisten yang membantu, jujur, dan aman.**

Pelatihan lanjutan setelah pretraining utama

Sebelum masuk post-training, banyak model melewati tahap tambahan yang masih bersifat pretraining:

- **Perpanjangan konteks (*long-context training*):** model dilatih dulu dengan konteks pendek (misalnya 8 ribu token) karena lebih murah, lalu dilatih lanjut dengan dokumen panjang agar bisa menangani konteks 128 ribu token atau lebih¹⁸.
- **Continued pretraining** pada domain tertentu (misalnya kode, kedokteran, atau bahasa tertentu) untuk menambah pengetahuan khusus.

Post-training: Dari Pembaca Internet Menjadi Asisten

Jika pretraining menghasilkan "otak" yang berpengetahuan, **post-training** membentuk "watak" dan "keterampilan"-nya. Tahap ini jauh lebih murah dalam komputasi dibanding pretraining (biasanya hanya sebagian kecil dari totalnya), tetapi dampaknya pada pengalaman pengguna sangat besar. Temuan InstructGPT tetap menjadi ilustrasi paling tajam: model 1,3 miliar parameter yang sudah dipoles lebih disukai manusia daripada GPT-3 mentah yang 100 kali lebih besar³³.

Resep post-training berkembang pesat. Secara historis dan konseptual, ada tiga lapis utama:

1. **Supervised Fine-Tuning (SFT)**: belajar dari contoh jawaban yang baik.
2. **Preference tuning**: belajar dari perbandingan jawaban mana yang lebih disukai (RLHF, DPO, RLAI/Constitutional AI).
3. **Reinforcement learning dengan hadiah terverifikasi**: berlatih memecahkan soal yang jawabannya bisa dicek, yang melahirkan *reasoning model*.

Dalam praktik modern, ketiga lapis ini sering dijalankan berulang-ulang dalam beberapa putaran, dicampur dengan pelatihan keamanan dan penggunaan alat.

I Tahap 1: Supervised Fine-Tuning (SFT)

Apa yang dilakukan

SFT, disebut juga *instruction tuning*, melanjutkan pelatihan base model dengan mekanisme yang **persis sama** seperti pretraining (tebak token berikutnya, backpropagation, gradient descent), tetapi pada **data yang sangat berbeda**: ribuan hingga jutaan contoh percakapan berformat "pertanyaan/instruksi → jawaban teladan".

Contoh data SFT:

Instruksi pengguna	Jawaban teladan
Jelaskan fotosintesis untuk anak SD	Tumbuhan itu seperti koki kecil. Mereka memakai sinar matahari, air, dan udara untuk membuat makanannya sendiri...
Tulis fungsi Python untuk membalik string	Berikut fungsinya, beserta penjelasan singkat...
Bagaimana cara membuat bahan peledak?	(Penolakan yang sopan, disertai penjelasan dan, jika relevan, alternatif yang aman)

Model dilatih agar ketika melihat bagian instruksi, ia menghasilkan bagian jawaban. Biasanya loss hanya dihitung pada bagian jawaban, karena kita ingin model belajar **menjawab**, bukan belajar meniru pertanyaan.

Format percakapan dan token khusus

Agar model bisa membedakan siapa yang berbicara, percakapan dibungkus dalam **template obrolan** dengan **token khusus** yang menandai awal dan akhir giliran, misalnya penanda "pesan sistem", "pengguna", dan "asisten". Saat Anda mengetik di aplikasi chat, sistem diam-diam membungkus pesan Anda dengan penanda ini, lalu meminta model melanjutkan teks setelah penanda "asisten". Model berhenti menulis saat ia menghasilkan token "akhir giliran". Jadi, bahkan chatbot terancang pun secara mekanis tetap **melanjutkan dokumen**; hanya saja dokumennya kini berbentuk transkrip percakapan, dan model sudah dilatih untuk memerankan pihak asisten.

Dari mana data SFT?

InstructGPT memakai sekitar 13 ribu contoh yang ditulis oleh pelabel manusia terlatih³³. Sekarang, data SFT sering berjumlah ratusan ribu hingga jutaan contoh, dan semakin banyak yang **dihasilkan atau disunting oleh model lain** lalu disaring secara otomatis dan manual. Pelajaran penting dari berbagai laporan teknis: **kualitas dan keragaman** data SFT jauh lebih menentukan daripada jumlahnya.

BAHASA MUDAH: SFT ADALAH MAGANG DENGAN CONTOH

Base model seperti lulusan baru yang sudah membaca semua buku tetapi belum pernah bekerja. SFT adalah masa magang di mana ia diperlihatkan ribuan contoh "beginilah cara karyawan yang baik membalas email pelanggan". Ia tidak diberi pengetahuan baru yang banyak; ia belajar **format, nada, dan kebiasaan** kerja. Karena itu, sebagian peneliti menyebut SFT terutama "membuka" kemampuan yang sudah ada sejak pretraining.

Fine-tuning hemat: LoRA

Melatih ulang seluruh parameter model raksasa itu mahal. **LoRA** (*Low-Rank Adaptation*, Hu dkk., 2021) membekukan bobot asli dan hanya melatih "tambalan" kecil yang disisipkan di setiap lapisan. Pada GPT-3 175B, LoRA mengurangi jumlah parameter yang dilatih hingga 10.000 kali dan kebutuhan memori GPU hingga 3 kali, dengan kualitas setara atau lebih baik daripada fine-tuning penuh⁴⁰. LoRA dan turunannya membuat perusahaan kecil, peneliti, dan bahkan hobiis bisa menyesuaikan model terbuka dengan satu atau beberapa GPU.

I Tahap 2: Belajar dari preferensi (RLHF, DPO, RLAIIF)

Mengapa SFT saja tidak cukup

SFT punya batasan mendasar:

- **Menulis jawaban sempurna itu sulit dan mahal**, sedangkan **menilai mana yang lebih baik dari dua jawaban** jauh lebih mudah. Seorang juri lomba masak tidak perlu bisa memasak lebih enak dari peserta untuk bisa menilai masakan mana yang lebih lezat.
- SFT hanya mengajarkan meniru. Ia tidak memberi tahu model **seberapa buruk** jawaban yang salah, atau mana dari dua jawaban yang sama-sama lumayan yang lebih disukai.
- Banyak kualitas yang diinginkan (membantu, jujur, tidak bertele-tele, aman) sulit didefinisikan dengan contoh saja, tetapi mudah dikenali saat dibandingkan.

RLHF klasik: tiga langkah InstructGPT

Reinforcement Learning from Human Feedback (RLHF), sebagaimana dipopulerkan InstructGPT (Ouyang dkk., 2022), berjalan dalam tiga langkah³³:

1. **SFT** seperti di atas, sebagai titik awal.
2. **Melatih reward model**. Untuk satu prompt, model menghasilkan beberapa jawaban. Pelabel manusia **mengurutkan** jawaban-jawaban itu dari terbaik ke terburuk. Data perbandingan ini dipakai melatih jaringan saraf terpisah, **reward model**, yang belajar memberi skor angka pada sebuah jawaban, memprediksi seberapa besar manusia akan menyukainya. InstructGPT memakai sekitar 33 ribu prompt untuk tahap ini³³.
3. **Optimasi dengan reinforcement learning**. Model bahasa (dalam istilah RL disebut *policy*) menghasilkan jawaban untuk prompt-prompt baru, reward model memberi skor, lalu algoritma RL bernama **PPO** (*Proximal Policy Optimization*)³⁴ menyetel parameter model agar jawaban berskor tinggi lebih sering muncul.

Satu komponen penting: **penalti KL**. Model diberi hukuman jika terlalu menyimpang dari model SFT awal. Tanpanya, model cenderung menemukan "celah" di reward model.

Reward hacking: ketika murid mengakali penilai

Reward model hanyalah tiruan selera manusia, dan tiruan itu tidak sempurna. Jika model dioptimalkan terlalu keras terhadapnya, ia akan menemukan cara mendapat skor tinggi tanpa benar-benar menjadi lebih baik, misalnya menulis jawaban lebih panjang, memakai nada sangat yakin, atau menyanjung pengguna. Fenomena ini disebut **reward hacking** atau *overoptimization*, dan Gao dkk. (2022) menunjukkan bahwa setelah titik tertentu, menaikkan skor reward model justru menurunkan kualitas yang sebenarnya⁴². Ini adalah versi AI dari **Hukum Goodhart**: "ketika sebuah ukuran dijadikan target, ia berhenti menjadi ukuran yang baik".

Salah satu gejala nyata adalah **sycophancy** (sikap menjilat): kecenderungan model membenarkan pendapat pengguna meskipun salah. Riset Anthropic (Sharma dkk., 2023) menemukan bahwa manusia penilai dan reward model sendiri kadang lebih menyukai jawaban yang meyakinkan dan sejalan dengan pandangan pengguna dibanding jawaban yang benar, sehingga RLHF bisa tanpa sengaja memperkuat kecenderungan ini⁴³.

DPO: RLHF tanpa RL

RLHF dengan PPO rumit dan rewel: harus menjalankan beberapa model sekaligus (policy, reward model, model referensi, dan model penilai nilai), dan pelatihannya mudah tidak stabil. **Direct Preference Optimization (DPO)** (Rafailov dkk., 2023) menunjukkan bahwa secara matematis, tujuan yang sama bisa dicapai **langsung** dari data pasangan "jawaban disukai vs tidak disukai", tanpa melatih reward model terpisah dan tanpa loop RL³⁵. Model cukup dilatih untuk menaikkan peluang jawaban yang disukai dan menurunkan peluang jawaban yang tidak disukai, relatif terhadap model awal. Judul makalahnya merangkum wawasan itu: "model bahasa Anda diam-diam adalah reward model". DPO dan variannya menjadi sangat populer, terutama di komunitas model terbuka, karena jauh lebih sederhana.

RLAIF dan Constitutional AI: umpan balik dari AI

Mengumpulkan jutaan penilaian manusia mahal, lambat, dan membebani para pelabel secara psikologis (terutama untuk konten berbahaya). **Constitutional AI** (Anthropic, 2022) menggantikan sebagian penilaian manusia dengan penilaian AI yang dipandu oleh daftar prinsip tertulis, sebuah "konstitusi"³⁶:

- **Fase terbimbing**: model menghasilkan jawaban, lalu diminta mengkritik jawabannya sendiri berdasarkan prinsip-prinsip itu dan merevisinya. Model dilatih pada versi yang sudah direvisi.
- **Fase RL**: model menghasilkan pasangan jawaban, dan sebuah model AI menilai mana yang lebih sesuai dengan konstitusi. Penilaian ini melatih *preference model*, yang lalu dipakai untuk RL. Metode umumnya disebut **RLAIF** (*RL from AI Feedback*).

Hasilnya adalah asisten yang tidak berbahaya tetapi juga tidak menghindar secara berlebihan: ia menjelaskan keberatannya alih-alih sekadar menolak³⁶. Pendekatan umpan balik AI semacam ini sekarang dipakai secara luas di industri dalam berbagai bentuk.

I Tahap 3: Reinforcement learning untuk penalaran

Chain-of-thought: berpikir di atas kertas

Pada 2022, Wei dkk. menunjukkan bahwa model besar menjawab soal matematika dan logika jauh lebih baik jika diminta menuliskan **langkah-langkah penalarannya** terlebih dahulu, sebuah teknik yang disebut **chain-of-thought (CoT)**³⁸. Intuisinya penting untuk memahami LLM: setiap

token dihasilkan dengan jumlah komputasi yang kurang lebih tetap. Soal yang sulit tidak bisa dipecahkan dalam "satu tarikan napas" (satu token jawaban). Dengan menulis langkah-langkah antara, model **menyebarkan komputasinya ke banyak token**, dan setiap langkah yang tertulis menjadi "catatan" yang bisa dibaca di langkah berikutnya lewat attention. Menulis adalah cara LLM berpikir.

Dari prompting ke pelatihan: reasoning model

Langkah berikutnya adalah **melatih** model agar terbiasa berpikir panjang dan efektif sebelum menjawab. OpenAI memperkenalkan **o1** pada September 2024, model yang dilatih dengan RL berskala besar untuk menghasilkan rantai penalaran panjang sebelum memberi jawaban. OpenAI melaporkan bahwa kinerjanya terus meningkat baik dengan lebih banyak komputasi RL saat pelatihan maupun dengan lebih banyak waktu "berpikir" saat menjawab³⁹. Detail metodenya tidak dipublikasikan.

Gambaran yang lebih terbuka datang dari **DeepSeek-R1** (Januari 2025; versi peer-review terbit di *Nature* September 2025)³⁷. Resepnya, disederhanakan:

1. Ambil base model yang kuat (DeepSeek-V3).
2. Kumpulkan banyak soal yang **jawabannya bisa diperiksa secara otomatis**: soal matematika dengan jawaban akhir pasti, soal pemrograman yang bisa diuji dengan *test case*, soal sains, teka-teki logika.
3. Untuk setiap soal, biarkan model menghasilkan **sekelompok** jawaban (misalnya 16 upaya), masing-masing dengan proses berpikirnya sendiri.
4. Beri hadiah (*reward*) berdasarkan **kebenaran jawaban akhir** (dan kepatuhan format), tanpa menilai langkah-langkahnya.
5. Perkuat upaya yang lebih baik dari rata-rata kelompoknya, lemahkan yang lebih buruk. Algoritma ini disebut **GRPO** (*Group Relative Policy Optimization*); ia tidak butuh model penilai terpisah karena "patokannya" adalah rata-rata skor kelompok itu sendiri³⁷.
6. Ulangi ribuan kali.

Temuan paling mencolok: pada varian **DeepSeek-R1-Zero**, yang dilatih dengan RL murni tanpa contoh penalaran buatan manusia, model secara spontan mulai menulis rantai berpikir yang makin panjang, **memeriksa ulang** pekerjaannya, mencoba pendekatan alternatif, dan mengoreksi diri, termasuk yang oleh penulisnya dijuluki "momen aha"³⁷. Tidak ada yang mengajarkan strategi itu secara eksplisit; strategi itu muncul karena terbukti menaikkan peluang mendapat hadiah. Versi akhir DeepSeek-R1 menggabungkan RL dengan sedikit data SFT awal dan beberapa putaran tambahan untuk memperbaiki keterbacaan dan perilaku umum.

BAHASA MUDAH: DARI MENIRU GURU KE LATIHAN SOAL

SFT seperti belajar dengan membaca contoh jawaban di buku. RLHF seperti mendapat komentar guru tentang jawaban mana yang lebih baik. RL dengan hadiah terverifikasi seperti mengerjakan ribuan soal latihan yang punya kunci jawaban di belakang buku: tidak ada yang mengajari **cara** mengerjakannya, tetapi siswa perlahan menemukan kebiasaan yang berhasil, misalnya menulis coretan, mengecek ulang, mencoba cara lain saat buntu. Karpathy menekankan bahwa di tahap inilah model bisa menemukan strategi yang **tidak ada** dalam data manusia, mirip AlphaGo yang menemukan langkah-langkah Go yang tak terpikirkan oleh pemain manusia⁶⁰.

Mengapa ini penting: dimensi skala baru

Reasoning model membuka dimensi penskalaan kedua. Selain memperbesar model dan data saat pelatihan (*train-time compute*), kinerja juga bisa ditingkatkan dengan memberi model lebih banyak waktu berpikir saat menjawab (*test-time compute* atau *inference-time scaling*)³⁹. Konsekuensinya, jawaban soal sulit menjadi lebih lambat dan lebih mahal, tetapi jauh lebih akurat. Keterbatasannya, RL semacam ini paling efektif pada domain yang hasilnya **bisa diverifikasi** (matematika, kode, sains). Untuk tugas terbuka seperti menulis esai atau memberi nasihat, "kunci jawaban" tidak ada, sehingga masih bergantung pada reward model yang rentan diakali.

| Pelatihan keamanan, kepribadian, dan penggunaan alat

Selain kemampuan, post-training juga membentuk:

- **Perilaku keamanan:** menolak permintaan yang benar-benar berbahaya (misalnya instruksi senjata biologis), tetapi tidak menolak berlebihan untuk permintaan wajar. Keseimbangan ini sulit; model yang terlalu hati-hati menjengkelkan, model yang terlalu longgar berbahaya.
- **Kejujuran dan kalibrasi:** mengakui ketidaktahuan, tidak mengarang sumber, menyatakan tingkat keyakinan.
- **Kepribadian dan gaya:** nada, panjang jawaban, format. Perbedaan "rasa" antar-chatbot sebagian besar berasal dari sini.
- **Penggunaan alat (*tool use*):** model dilatih menghasilkan token khusus yang berarti "panggil kalkulator / mesin pencari / jalankan kode", lalu hasilnya disisipkan kembali ke konteks. Toolformer (2023) menunjukkan model bahkan bisa belajar sendiri kapan alat berguna⁴⁴. Kemampuan ini adalah fondasi **agen AI** yang bisa menjelajah web, menulis dan menjalankan program, atau mengoperasikan komputer.

Distilasi: guru besar, murid kecil

Satu teknik yang melintasi pretraining dan post-training adalah **distilasi pengetahuan** (*knowledge distillation*, Hinton dkk., 2015): model kecil ("murid") dilatih untuk meniru keluaran model besar ("guru")⁴¹. Murid tidak hanya belajar dari jawaban akhir guru, tetapi bisa juga dari distribusi peluang lengkapnya, yang membawa lebih banyak informasi ("jawaban A 70%, B 25%" lebih informatif daripada sekadar "A"). DeepSeek, misalnya, merilis beberapa model kecil yang didistilasi dari R1 dan mewarisi sebagian kemampuan penalarannya³⁷. Banyak model kecil yang sangat kuat saat ini adalah hasil distilasi dari model yang jauh lebih besar.

Ringkasan post-training

Metode	Data yang dibutuhkan	Apa yang diajarkan	Kelebihan	Kelemahan
SFT	Contoh instruksi → jawaban teladan	Format, nada, mengikuti instruksi	Sederhana, stabil	Hanya meniru; mahal membuat jawaban sempurna
RLHF (reward model + PPO)	Peringkat/perbandingan jawaban oleh manusia	Selera manusia yang sulit didefinisikan	Kuat, fleksibel	Rumit, tidak stabil, rawan reward hacking
DPO	Pasangan jawaban disukai/tidak	Sama seperti RLHF	Jauh lebih sederhana	Kurang bisa "menjelajah" jawaban baru
Constitutional AI / RLAI	Prinsip tertulis + penilaian AI	Keamanan & nilai	Murah, skalabel, transparan prinsipnya	Bergantung pada kualitas penilai AI
RL hadiah terverifikasi (misal GRPO)	Soal dengan jawaban yang bisa dicek	Penalaran, strategi pemecahan masalah	Bisa melampaui contoh manusia	Terbatas pada domain yang bisa diverifikasi

Evaluasi: Bagaimana Tahu Modelnya Bagus?

Membuat model tanpa mengukurnya seperti memasak tanpa mencicipi. Evaluasi dilakukan di sepanjang proses, bukan hanya di akhir.

I Jenis-jenis evaluasi

- **Loss / perplexity** pada data validasi: ukuran paling mentah selama pretraining. Turunnya loss adalah tanda model belajar, tetapi tidak langsung menjawab "apakah model berguna?".
- **Benchmark otomatis**: kumpulan soal standar dengan jawaban pasti, misalnya soal pengetahuan pilihan ganda lintas bidang, matematika kompetisi, pemrograman (misalnya memperbaiki *issue* nyata di repositori GitHub), sains tingkat doktoral. Stanford AI Index 2026 mencatat bahwa pada tolok ukur SWE-bench Verified, skor melonjak dari sekitar 60% menjadi mendekati 100% hanya dalam setahun, dan model terdepan kini menyamai atau melampaui patokan manusia pada soal sains tingkat doktoral dan matematika kompetisi⁵⁸.
- **Evaluasi preferensi manusia**: manusia membandingkan jawaban dua model secara buta. Platform seperti LMArena (dulu Chatbot Arena) mengumpulkan jutaan penilaian semacam ini dan menyusun peringkat mirip sistem skor catur.
- **Evaluasi keamanan dan red-teaming**: tim khusus (manusia dan AI) mencoba memancing model melakukan hal berbahaya: memberi instruksi senjata, membantu serangan siber, membocorkan data, atau lolos dari batasan (*jailbreak*). Laboratorium besar juga menguji "kemampuan berbahaya" sebelum rilis.

I Masalah-masalah evaluasi

- **Kontaminasi data**: soal benchmark yang sudah beredar di internet bisa ikut masuk data pretraining, sehingga model "hafal" soalnya. Skor tinggi belum tentu berarti kemampuan umum.
- **Benchmark cepat jenuh**: begitu sebuah tolok ukur dianggap penting, semua lab mengoptimalkan diri untuknya, dan skornya mendekati 100% dalam beberapa tahun. Komunitas terus membuat tolok ukur baru yang lebih sulit.
- **Kemampuan yang tidak merata**: AI Index 2026 menyoroti bahwa model yang bisa menang olimpiade matematika masih kesulitan dengan tugas yang tampak sepele, misalnya membaca jam analog⁵⁸. Kemampuan LLM sering disebut "**bergerigi**" (*jagged*): luar biasa di satu hal, mengejutkan lemahnya di hal lain.

Inferensi: Apa yang Terjadi Saat Anda Bertanya

Setelah dilatih, parameter model **dibekukan**. Menjalankan model untuk menghasilkan jawaban disebut **inferensi**. Penting dicatat: saat Anda mengobrol dengan chatbot, **model tidak sedang belajar**. Tidak ada backpropagation, tidak ada parameter yang berubah. Semua yang "diingat" model tentang percakapan Anda hanyalah teks percakapan itu sendiri yang terus dikirim ulang sebagai konteks.

I Langkah demi langkah

1. **Susun prompt lengkap**: sistem menggabungkan pesan sistem (instruksi tersembunyi dari penyedia layanan atau pengembang aplikasi), riwayat percakapan, dokumen lampiran, hasil pencarian, dan pesan terbaru Anda, lalu membungkusnya dengan template obrolan.
2. **Tokenisasi** seluruh teks itu.
3. **Prefill**: seluruh token prompt diproses sekaligus secara paralel lewat semua lapisan. Tahap ini menentukan berapa lama sebelum kata pertama muncul.
4. **Decode**: model menghasilkan satu token, menambahkannya ke urutan, lalu menghitung token berikutnya. Satu per satu, sampai muncul token "akhir giliran" atau batas panjang tercapai. Inilah sebabnya jawaban tampak "diketik" di layar: token dikirim begitu dihasilkan (*streaming*).
5. **Detokenisasi**: token diubah kembali menjadi teks.

I Sampling: memilih dari distribusi peluang

Di setiap langkah, model mengeluarkan distribusi peluang atas seluruh kosakata. Cara memilih satu token darinya disebut **strategi decoding** atau **sampling**:

- **Greedy**: selalu pilih token berpeluang tertinggi. Deterministik, tetapi cenderung menghasilkan teks datar, repetitif, bahkan berputar-putar⁴⁵.
- **Temperature**: "kenop kreativitas". Temperatur rendah mempertajam distribusi (pilihan lebih aman dan konsisten); temperatur tinggi meratakan distribusi (pilihan lebih beragam, tetapi lebih rawan ngawur).
- **Top-k dan top-p (nucleus sampling)**: hanya ambil sampel dari sekelompok token teratas, misalnya token-token yang peluang kumulatifnya mencapai 90%, sehingga token-token absurd di "ekor" distribusi tidak pernah terpilih⁴⁵.

Karena ada unsur acak, pertanyaan yang sama bisa menghasilkan jawaban berbeda. Ini bukan *bug*; ini sifat bawaan.

I KV cache: tidak menghitung ulang yang sudah lewat

Saat menghasilkan token ke-501, model memerlukan key dan value dari 500 token sebelumnya. Menghitung ulang semuanya di setiap langkah sangat boros. Maka key dan value yang sudah dihitung disimpan di memori GPU, disebut **KV cache**. Masalahnya, KV cache untuk konteks panjang dan ribuan pengguna sekaligus bisa sangat besar, sering kali lebih besar dari modelnya sendiri. Banyak inovasi inferensi berpusat pada pengelolaan memori ini, misalnya **PagedAttention** pada sistem vLLM yang meminjam ide memori virtual dari sistem operasi untuk mengurangi pemborosan memori⁴⁶, serta GQA dan varian attention lain yang memperkecil ukuran cache²²²⁰.

I Jendela konteks (*context window*)

Jendela konteks adalah jumlah maksimum token yang bisa "dilihat" model sekaligus: prompt, riwayat, lampiran, dan jawaban. Ini adalah **memori kerja** model. Apa pun di luar jendela konteks, bagi model, tidak ada. Jendela konteks model terdepan tumbuh dari 2.048 token (GPT-3, 2020)¹⁷ menjadi ratusan ribu hingga sekitar satu juta token pada banyak model tahun 2025–2026.

Namun konteks panjang tidak berarti model memakai semuanya dengan sama baik. Riset "Lost in the Middle" (2023) menemukan bahwa model cenderung paling baik memanfaatkan informasi di **awal dan akhir** konteks, dan lebih sering melewati informasi di tengah⁴⁹. Model-model baru banyak memperbaiki hal ini, tetapi prinsipnya tetap berlaku: konteks yang ringkas dan relevan biasanya menghasilkan jawaban lebih baik.

I Membuat inferensi lebih murah dan cepat

- **Kuantisasi** (*quantization*): menyimpan bobot dengan presisi lebih rendah (misalnya 8 atau 4 bit, bukan 16 bit). Model menjadi dua sampai empat kali lebih kecil dan lebih cepat, dengan penurunan kualitas yang biasanya kecil. Inilah yang memungkinkan model terbuka dijalankan di laptop atau bahkan ponsel.
- **Batching**: melayani banyak pengguna sekaligus dalam satu perhitungan agar GPU tidak menganggur.
- **Speculative decoding**: model kecil yang cepat "menebak" beberapa token ke depan, lalu model besar memeriksa semuanya sekaligus dalam satu langkah. Token yang disetujui diterima, yang salah dibuang. Hasilnya identik dengan model besar, tetapi lebih cepat⁴⁷.
- **Distilasi dan MoE**, yang sudah dibahas, juga pada dasarnya adalah strategi menekan biaya inferensi.

I Memperluas kemampuan saat inferensi: RAG, alat, dan agen

Karena pengetahuan model beku pada saat pelatihan (ada **batas pengetahuan** atau *knowledge cutoff*), dan karena model bisa keliru mengingat fakta, sistem LLM modern sering dilengkapi:

- **RAG** (*Retrieval-Augmented Generation*): sebelum menjawab, sistem mencari dokumen relevan (dari web atau basis data internal perusahaan) dan menyisipkannya ke konteks, sehingga model menjawab berdasarkan teks yang ada di depannya, bukan hanya dari ingatan parameter⁴⁸.
- **Pemanggilan alat**: kalkulator, penerjemah kode, pencarian web, API lain.
- **Agen**: model yang menjalankan loop "rencanakan → bertindak (panggil alat) → amati hasil → rencanakan lagi" untuk menyelesaikan tugas bertahap, misalnya menulis perangkat lunak, meriset topik, atau mengisi formulir. Asisten pemrograman yang bisa membaca seluruh repositori, menjalankan tes, dan memperbaiki kode sendiri adalah contoh agen yang kini banyak dipakai.

IMPLIKASI PRAKTIS BAGI PENGGUNA

- Model tidak mengingat percakapan lain kecuali aplikasi sengaja menyimpan dan menyisipkan "memori" ke konteks.
- Menyertakan dokumen sumber di prompt biasanya menghasilkan jawaban lebih akurat daripada mengandalkan ingatan model.
- Untuk soal rumit, meminta model menjelaskan langkah demi langkah (atau memakai mode "berpikir") memberi model lebih banyak ruang komputasi.
- Untuk fakta penting, angka, kutipan, dan rujukan: selalu verifikasi.

Resep Lengkap: Membangun LLM dari Nol

Bagian ini merangkai semua tahap menjadi satu alur utuh, seolah-olah kita adalah tim yang ditugasi membuat LLM baru. Tujuannya menunjukkan bagaimana setiap keputusan saling terkait.

I Langkah 0: Tentukan tujuan dan anggaran

Semuanya berawal dari pertanyaan: model ini untuk apa, dan berapa anggaran komputasinya? Model kecil untuk ponsel, model menengah untuk perusahaan, atau model terdepan? Dengan hukum penskalaan²³²⁴, tim memperkirakan kombinasi ukuran model dan jumlah token yang masuk akal untuk anggaran itu, sambil mempertimbangkan biaya inferensi di masa depan (yang mendorong model lebih kecil dengan data lebih banyak).

I Langkah 1: Bangun korpus data

Kumpulkan teks web, kode, buku, makalah, data matematika, dan multibahasa. Bersihkan: ekstraksi teks, filter bahasa, filter kualitas heuristik dan berbasis model, deduplikasi, hapus data pribadi, dekontaminasi benchmark. Tentukan proporsi campuran lewat eksperimen pada model kecil. Hasilnya belasan triliun token siap latih²⁵¹⁸.

I Langkah 2: Latih tokenizer

Jalankan BPE pada sampel korpus untuk menghasilkan kosakata puluhan hingga lebih dari seratus ribu token, dengan perhatian khusus pada angka, kode, dan bahasa non-Inggris¹². Setelah ditetapkan, tokenizer tidak bisa diubah tanpa melatih ulang model.

I Langkah 3: Rancang arsitektur

Pilih Transformer decoder-only: berapa blok, seberapa lebar vektor, berapa kepala attention, dense atau MoE, jenis attention (misalnya GQA), posisi (RoPE), jenis normalisasi dan aktivasi, panjang konteks awal¹¹³²¹²². Sebagian besar pilihan ini diuji terlebih dahulu pada model kecil.

I Langkah 4: Siapkan infrastruktur

Sewa atau bangun kluster ribuan GPU dengan jaringan berkecepatan sangat tinggi. Siapkan kerangka pelatihan terdistribusi (paralelisme data, tensor, pipeline, ZeRO/FSDP), presisi campuran, checkpointing otomatis, dan sistem pemantauan²⁹³⁰³¹.

I Langkah 5: Pretraining

Inisialisasi semua parameter secara acak. Jalankan loop: ambil batch → umpan maju → hitung loss → backpropagation → pembaruan AdamW, dengan jadwal learning rate warmup lalu decay²¹⁰¹¹. Pantau loss dan benchmark kecil secara berkala. Tangani kegagalan perangkat keras dan loss spike. Di akhir, lakukan annealing dengan data berkualitas tinggi, lalu perpanjang konteks. Proses ini berlangsung berminggu-minggu hingga berbulan-bulan dan menghasilkan base model.

I Langkah 6: Post-training

Mulai dengan SFT pada data instruksi berkualitas. Lanjutkan dengan preference tuning (RLHF, DPO, atau RLAIFF) untuk membantu, jujur, dan aman. Tambahkan RL dengan hadiah terverifikasi untuk matematika, kode, dan penalaran. Latih penggunaan alat dan perilaku agen. Ulangi beberapa putaran, sering kali memakai model hasil putaran sebelumnya untuk membuat data putaran berikutnya³³³⁵³⁶³⁷.

I Langkah 7: Evaluasi dan red-teaming

Uji dengan benchmark kemampuan, preferensi manusia, evaluasi keamanan, dan red-teaming. Perbaiki kelemahan yang ditemukan dengan data dan pelatihan tambahan.

I Langkah 8: Optimasi dan penerapan

Kuantisasi, siapkan sistem penyajian (KV cache, batching, speculative decoding), tambahkan lapisan pengaman di luar model (filter masukan/keluaran), sediakan API dan aplikasi, lalu pantau penggunaan nyata. Umpan balik pengguna (misalnya tombol jempol naik/turun) menjadi bahan untuk versi berikutnya.

SIAPA YANG BISA MELAKUKANNYA?

Membuat model terdepan membutuhkan ratusan juta hingga miliaran dolar, ribuan GPU, dan tim ratusan orang. Namun **prinsipnya** bisa dipelajari dan dipraktikkan dalam skala kecil: proyek pendidikan seperti nanoGPT dan kuliah "Neural Networks: Zero to Hero" dari Andrej Karpathy menunjukkan cara melatih model mirip GPT-2 dari nol dengan kode beberapa ratus baris. Sementara itu, menyesuaikan model terbuka yang sudah ada dengan LoRA bisa dilakukan dengan satu GPU⁴⁰.

Batasan dan Miskonsepsi

Memahami cara kerja LLM langsung menjelaskan banyak kelemahannya. Kelemahan-kelemahan ini bukan kebetulan, melainkan konsekuensi dari desainnya.

I Halusinasi

Halusinasi adalah ketika model menghasilkan informasi yang terdengar meyakinkan tetapi salah: fakta keliru, kutipan palsu, rujukan yang tidak pernah ada. Mengapa ini terjadi?

- **Model dilatih untuk menghasilkan teks yang masuk akal, bukan teks yang benar.** Selama pretraining, "benar" hanya berarti "sesuai dengan teks yang ada di data". Untuk fakta yang jarang muncul di data (misalnya tanggal lahir orang yang tidak terkenal), model tidak punya ingatan yang kuat, tetapi tetap harus menghasilkan token yang berbentuk tanggal.
- **Evaluasi dan pelatihan cenderung menghargai tebakan.** Makalah OpenAI tahun 2025, "Why Language Models Hallucinate", berargumen bahwa sebagian besar tolok ukur menilai jawaban secara benar/salah: menjawab "tidak tahu" mendapat nol, sedangkan menebak punya peluang dapat nilai. Akibatnya, model dioptimalkan menjadi "peserta ujian yang pandai menebak"⁵². Mereka mengusulkan agar sistem penilaian memberi ganjaran pada ketidakpastian yang jujur.
- **Pengetahuan tersimpan secara tersebar dan *lossy*.** Model tidak punya "database" untuk dicek. Karpathy memakai analogi: pengetahuan dalam parameter itu seperti **ingatan samar** tentang sesuatu yang dibaca bulan lalu, sedangkan informasi di jendela konteks itu seperti **memori kerja**, catatan yang ada tepat di depan mata⁶⁰.

Riset interpretabilitas Anthropic (2025) menemukan petunjuk menarik tentang mekanisme ini pada Claude: model tampaknya punya sirkuit bawaan untuk menolak menjawab ketika tidak mengenal sesuatu, dan halusinasi bisa terjadi ketika sirkuit "saya mengenal ini" salah menyala untuk hal yang sebenarnya tidak diketahui⁵¹.

Miskonsepsi umum

Miskonsepsi	Kenyataan
"LLM mencari jawaban di internet."	Model murni hanya memakai parameternya. Pencarian web adalah alat tambahan yang dipasang di sekitar model (RAG).
"LLM menyimpan salinan teks yang dibacanya."	Pengetahuan tersimpan dalam bentuk bobot yang tersebar. Namun teks yang sering berulang bisa terhafal hampir persis, sehingga hafalan tetap terjadi secara parsial.
"LLM belajar dari percakapan saya."	Saat inferensi, parameter tidak berubah. Percakapan bisa dipakai perusahaan sebagai data pelatihan versi berikutnya (tergantung kebijakan dan pengaturan privasi), tetapi itu proses terpisah.
"LLM itu hanya autocomplete, jadi tidak mungkin bernalar."	Tujuan pelatihannya memang menebak token, tetapi untuk menebak dengan baik model membangun mekanisme internal yang kompleks. Riset menunjukkan, misalnya, bahwa model merencanakan kata rima beberapa kata ke depan saat menulis puisi ⁵¹ . Apakah itu "penalaran sejati" adalah perdebatan filosofis.
"Semakin yakin nada jawabannya, semakin benar."	Nada yakin sebagian adalah hasil gaya yang diperkuat post-training. Keyakinan verbal tidak selalu terkalibrasi dengan kebenaran.
"Model tahu tanggal hari ini dan kejadian terbaru."	Model hanya tahu hingga batas pengetahuannya, kecuali tanggal dan informasi baru disisipkan ke konteks.

Batasan lain

- **Sycophancy**: cenderung membenarkan pengguna⁴³.
- **Bias**: model mewarisi bias dari data internet dan dari pilihan pelabel. Post-training mengurangi sebagian, tetapi tidak menghapus sepenuhnya.
- **Kemampuan tidak merata antarbahasa**: bahasa dengan data sedikit mendapat kinerja lebih rendah dan biaya token lebih tinggi.
- **Kerentanan terhadap manipulasi prompt**: *jailbreak* dan *prompt injection* (instruksi jahat yang disembunyikan dalam dokumen atau halaman web yang dibaca model), yang menjadi masalah serius untuk agen yang bisa bertindak.
- **Kotak hitam**: kita tahu persis operasi matematis yang dijalankan model, tetapi belum sepenuhnya memahami **apa** yang direpresentasikan oleh miliaran parameternya. Bidang **interpretabilitas mekanistik** berusaha membongkarnya; Anthropic, misalnya, berhasil mengekstrak jutaan "fitur" yang dapat dipahami dari Claude 3 Sonnet, mulai dari konsep kota dan tokoh terkenal hingga konsep abstrak seperti kerentanan keamanan pada kode, dan menunjukkan bahwa menyetel fitur-fitur itu mengubah perilaku model⁵⁰.

Data & Angka Kunci

Skala model-model tonggak (hanya yang datanya dipublikasikan)

Model	Tahun	Parameter	Token pretraining	Catatan	Sumber
Transformer (base/big)	2017	±65 juta / ±213 juta	(data terjemahan WMT)	Model penerjemah, bukan LLM generatif	1
GPT-1	2018	±117 juta	(BooksCorpus)	Membuktikan resep pretraining + fine-tuning	14
GPT-2	2019	1,5 miliar	(WebText, ±40 GB teks)	Kemampuan tanpa pelatihan khusus (<i>zero-shot</i>)	16
GPT-3	2020	175 miliar	±300 miliar	Few-shot learning; konteks 2.048 token	17
Chinchilla	2022	70 miliar	1,4 triliun	Membuktikan pentingnya data (±20 token/parameter)	24
Llama 3.1 405B	2024	405 miliar (dense)	±15,6 triliun	±16 ribu GPU H100; 30,84 juta jam-GPU	1819
Mixtral 8x7B	2024	46,7 miliar (±13 miliar aktif)	tidak diungkap	MoE, 2 dari 8 expert per token	21
DeepSeek-V3	2024	671 miliar (37 miliar aktif)	14,8 triliun	MoE; 2,788 juta jam-GPU H800; pelatihan FP8	20
DeepSeek-R1	2025	(berbasis V3)	–	Penalaran lewat RL (GRPO)	37

Tren komputasi, biaya, dan sumber daya

Indikator	Angka	Tahun data	Sumber
Pertumbuhan komputasi pelatihan model terdepan	$\pm 4\text{--}5\times$ per tahun	2010–2024	Epoch AI ²⁸
Biaya komputasi pelatihan Transformer asli	$\pm$ US\$670	2017 (estimasi)	AI Index 2025 ⁵⁷
Biaya komputasi pelatihan GPT-4	$\pm$ US\$78 juta	2023 (estimasi)	AI Index 2025 ⁵⁷
Biaya komputasi pelatihan Gemini Ultra	$\pm$ US\$192 juta	2023 (estimasi)	AI Index 2025 ⁵⁷
Biaya "run akhir" DeepSeek-V3 (asumsi US\$2/jam-GPU)	$\pm$ US\$5,6 juta, di luar biaya riset & perangkat	2024	DeepSeek ²⁰
Emisi pelatihan Llama 3.1 (berbasis lokasi)	± 11.390 ton CO ₂ e (seluruh keluarga)	2024	Meta ¹⁹
Emisi pelatihan Grok 4 (estimasi)	± 72.816 ton CO ₂ e	2025	AI Index 2026 ⁵⁸
Kapasitas daya pusat data AI	$\pm 29,6$ GW	2025	AI Index 2026 ⁵⁸
Investasi AI korporasi global	$\pm$ US\$581,7 miliar (naik 130%)	2025	AI Index 2026 ⁵⁸

Data dan batasnya

Indikator	Angka	Sumber
Ukuran FineWeb (web Inggris yang dibersihkan)	± 15 triliun token dari 96 arsip Common Crawl	Hugging Face 2024 ²⁵
FineWeb-Edu (subset edukatif)	$\pm 1,3$ triliun token	Hugging Face 2024 ²⁵
Perkiraan stok teks publik buatan manusia	± 300 triliun token	Villalobos dkk. 2024 ²⁶
Perkiraan kapan data publik "habis" terpakai	antara 2026 dan 2032 jika tren berlanjut	Villalobos dkk. 2024 ²⁶

Post-training: angka dari InstructGPT

Komponen	Angka	Sumber
Contoh SFT (ditulis pelabel)	±13 ribu prompt	33
Prompt untuk melatih reward model	±33 ribu	33
Hasil utama	InstructGPT 1,3 miliar parameter lebih disukai daripada GPT-3 175 miliar	33
LoRA pada GPT-3 175B	parameter yang dilatih turun hingga 10.000×, memori GPU turun 3×	40

MEMBACA ANGKA BIAYA DENGAN HATI-HATI

Estimasi biaya pelatihan biasanya hanya menghitung **komputasi untuk satu run pelatihan akhir**, bukan gaji peneliti, ratusan eksperimen gagal, pengumpulan data, pelabelan manusia, atau pembelian kluster. Biaya total pengembangan model terdepan bisa berkali-kali lipat lebih besar. Sebaliknya, angka yang sangat kecil (seperti US\$5,6 juta untuk DeepSeek-V3) sering dikutip tanpa catatan kaki penting ini.

Perdebatan & Perspektif

I Apakah LLM "memahami"? Burung beo stokastik vs model dunia

Ini perdebatan paling mendasar, dan belum selesai.

Pandangan skeptis. Makalah "On the Dangers of Stochastic Parrots" (Bender, Gebru, dkk., 2021) berargumen bahwa model bahasa hanya merangkai bentuk linguistik berdasarkan pola statistik, tanpa akses pada makna, karena yang mereka lihat hanyalah teks, bukan dunia yang dirujuk teks itu. Istilah "**burung beo stokastik**" menjadi slogan kubu ini. Makalah itu juga menyoroti biaya lingkungan, bias dalam data, dan bahaya teks yang tampak meyakinkan tanpa tanggung jawab⁵³.

Pandangan sebaliknya. Sejumlah riset menunjukkan bahwa model yang dilatih hanya dengan prediksi token bisa membentuk **representasi internal tentang dunia**. Contoh terkenal adalah **Othello-GPT**: model kecil yang dilatih hanya dengan urutan langkah permainan Othello, tanpa diberi tahu aturan atau bentuk papan, ternyata menyimpan representasi keadaan papan di dalam aktivasinya, dan mengubah representasi itu mengubah langkah yang diprediksinya⁵⁴. Riset interpretabilitas pada model besar menemukan fitur-fitur abstrak yang konsisten lintas bahasa dan modalitas⁵⁰, serta bukti perencanaan ke depan⁵¹.

Posisi tengah. Banyak peneliti kini berpendapat bahwa pertanyaan "memahami atau tidak" terlalu biner. LLM jelas membentuk representasi yang lebih dari sekadar statistik permukaan, tetapi representasi itu bisa rapuh, tidak konsisten, dan berbeda dari pemahaman manusia yang terhubung dengan tubuh, indra, dan pengalaman. Kemampuan yang "bergerigi" (luar biasa di satu hal, gagal di hal sederhana) adalah bukti bahwa kecerdasan LLM bukan salinan kecerdasan manusia.

I Kemampuan emergen: nyata atau ilusi pengukuran?

Wei dkk. (2022) mendokumentasikan **kemampuan emergen** (*emergent abilities*): kemampuan yang tampak tidak ada pada model kecil lalu tiba-tiba muncul setelah skala tertentu, misalnya aritmetika multi-digit atau chain-of-thought⁵⁵. Ini memicu kekhawatiran bahwa kemampuan (termasuk yang berbahaya) bisa muncul tanpa peringatan.

Schaeffer dkk. (2023) membantah sebagian klaim itu: menurut mereka, "lompatan" sering merupakan artefak dari **cara mengukur**. Jika skor dihitung dengan metrik semua-atau-tidak-sama-sekali (jawaban harus persis benar), peningkatan yang sebenarnya mulus terlihat seperti lompatan. Dengan metrik yang lebih halus, kurvanya menjadi landai⁵⁶. Perdebatan ini penting untuk kebijakan: jika kemampuan tumbuh mulus, ia lebih mudah diprediksi dan diawasi.

I Apakah penskalaan akan terus berhasil?

- **Kubu optimis** menunjuk pada tren yang konsisten selama bertahun-tahun, lonjakan skor tolok ukur (AI Index 2026 menyebut kemampuan "berakselerasi, bukan mendatar")⁵⁸, dan dimensi skala baru berupa test-time compute dan RL³⁹³⁷.
- **Kubu skeptis** menunjuk pada biaya yang tumbuh eksponensial, keterbatasan listrik, hasil yang makin mahal per peningkatan, dan kelemahan yang belum hilang (halusinasi, penalaran yang rapuh di luar distribusi latihan). Sebagian berpendapat bahwa arsitektur atau paradigma baru dibutuhkan untuk mencapai kecerdasan setara manusia.

I Apakah data akan habis? Data sintetis dan "keruntuhan model"

Villalobos dkk. memperkirakan stok teks publik buatan manusia sekitar 300 triliun token, dan dengan tren saat ini, model akan dilatih dengan dataset sebesar seluruh stok itu antara 2026 dan 2032²⁶. Jalan keluar yang dibahas meliputi data sintetis, data multimodal (video, audio), efisiensi data yang lebih baik, dan RL (yang menghasilkan datanya sendiri lewat latihan).

Namun data sintetis punya risiko. Shumailov dkk. (*Nature*, 2024) menunjukkan bahwa melatih model secara berulang pada keluaran model generasi sebelumnya tanpa seleksi menyebabkan **model collapse**: variasi di "ekor" distribusi perlahan hilang, dan model makin seragam dan menyimpang²⁷. Riset lanjutan menunjukkan bahwa risiko ini bisa dikurangi jika data sintetis **dicampur** dengan data asli dan **disaring** dengan cermat, dan dalam praktik, data sintetis yang dikurasi dengan baik sudah menjadi bagian penting pelatihan model modern.

I Model terbuka vs tertutup

- **Model tertutup** (*closed-weight*) hanya bisa diakses lewat API atau aplikasi. Pendukungnya berargumen bahwa ini memungkinkan kontrol keamanan dan mencegah penyalahgunaan.
- **Model terbuka** (*open-weight*, misalnya Llama, DeepSeek, Qwen, Mistral) merilis parameternya sehingga siapa pun bisa menjalankan dan memodifikasinya. Pendukungnya berargumen bahwa ini mendorong riset, transparansi, kedaulatan teknologi, dan persaingan. Laporan teknis model terbuka (Llama 3, DeepSeek-V3/R1) adalah sumber utama pengetahuan publik tentang cara membuat LLM, termasuk dokumen ini.
- Pertanyaan keamanannya nyata: model terbuka tidak bisa "ditarik kembali" dan pengamanan bawaannya bisa dilepas dengan fine-tuning. Perdebatan ini bersinggungan dengan geopolitik; AI Index 2026 mencatat bahwa model-model AS dan Tiongkok saling bergantian memimpin sejak awal 2025, dan per Maret 2026 selisih kinerja teratas hanya sekitar 2,7%⁵⁸.

I Etika, hak cipta, dan dampak sosial

- **Hak cipta dan persetujuan:** data pretraining mencakup karya berhak cipta yang dikumpulkan tanpa izin eksplisit. Sejumlah gugatan hukum dari penulis, penerbit, dan media sedang atau telah berjalan di berbagai negara, dan hasilnya akan menentukan aturan main pengumpulan data ke depan.
- **Energi dan lingkungan:** kebutuhan listrik pusat data AI tumbuh pesat⁵⁸²⁸.
- **Pekerjaan:** AI Index 2026 mencatat penurunan hampir 20% lapangan kerja pengembang perangkat lunak usia 22–25 tahun sejak 2024 di bidang yang mengalami peningkatan produktivitas AI, meskipun hubungan sebab-akibatnya masih diperdebatkan⁵⁸.
- **Transparansi:** seperti disebut di awal, keterbukaan laboratorium besar justru menurun⁵⁸.

Kondisi Terkini & Prospek

Keadaan per September 2026.

I Keadaan saat ini

- **Resep dasar tetap sama.** Hampir semua LLM terdepan masih berupa Transformer yang di-pretrain dengan prediksi token berikutnya, lalu di-post-train. Inovasinya lebih banyak di skala, data, efisiensi (MoE, attention yang lebih hemat, presisi rendah), dan post-training.
- **Post-training dan RL menjadi medan persaingan utama.** Sejak 01 (2024) dan DeepSeek-R1 (2025), model-model terdepan dari berbagai laboratorium umumnya menawarkan mode "berpikir" yang memakai rantai penalaran panjang sebelum menjawab³⁹³⁷.
- **Konteks panjang menjadi standar.** Banyak model terdepan mendukung konteks ratusan ribu hingga sekitar satu juta token.
- **Dari chatbot ke agen.** Penggunaan bergeser dari tanya-jawab ke agen yang mengerjakan tugas multi-langkah: menulis dan menguji kode, riset, dan mengoperasikan perangkat lunak. Lonjakan skor SWE-bench Verified menuju hampir 100% dalam setahun mencerminkan pergeseran ini⁵⁸.
- **Adopsi masif dan investasi raksasa.** AI generatif mencapai 53% adopsi populasi dalam tiga tahun, dan investasi AI korporasi global mencapai sekitar US\$581,7 miliar pada 2025⁵⁸.
- **Persaingan global yang ketat.** Selisih kinerja model teratas AS dan Tiongkok menyempit hingga beberapa persen saja⁵⁸, didorong sebagian oleh model-model terbuka dari Tiongkok yang efisien.

I Tren dan pertanyaan ke depan

1. **Penskalaan RL.** Seberapa jauh RL bisa diperluas dari domain yang terverifikasi (matematika, kode) ke tugas terbuka (riset, penulisan, keputusan bisnis)? Ini mungkin pertanyaan teknis terpenting beberapa tahun ke depan.
2. **Data.** Dengan data teks publik mendekati batas²⁶, sumber pertumbuhan akan bergeser ke data sintesis yang terkurasi, data multimodal, data interaksi agen, dan efisiensi belajar.
3. **Efisiensi.** Model kecil hasil distilasi dan kuantisasi yang berjalan di perangkat pribadi akan makin mampu, sementara model terbesar tetap di pusat data.
4. **Interpretabilitas dan keamanan.** Kemampuan membaca "pikiran" model (fitur, sirkuit) berkembang pesat⁵⁰⁵¹, dan akan menentukan seberapa besar kita bisa memercayai sistem yang semakin otonom.
5. **Energi dan infrastruktur.** Kebutuhan daya yang tumbuh cepat²⁸⁵⁸ menjadi kendala fisik dan politik yang nyata.

SKENARIO, BUKAN RAMALAN

Skenario optimis: RL dan agen membuat LLM menjadi rekan kerja riset dan rekayasa yang andal, dengan interpretabilitas yang cukup untuk mengawasinya. Skenario moderat: kemajuan berlanjut tetapi melambat karena biaya, data, dan energi, dan LLM menjadi infrastruktur seperti listrik atau internet, bukan kecerdasan super. Skenario pesimis: kelemahan mendasar (halusinasi, kerentanan manipulasi, reward hacking) terbukti sulit diatasi pada sistem yang makin otonom, sehingga adopsi tersendat dan regulasi menegat. Bukti hingga September 2026 belum cukup untuk memastikan skenario mana yang akan terjadi.

Kronologi

Tahun	Peristiwa
1970	Linnainmaa menerbitkan <i>reverse-mode automatic differentiation</i> , inti matematis backpropagation ³
1986	Rumelhart, Hinton & Williams mempopulerkan backpropagation untuk jaringan saraf (<i>Nature</i>) ²
1997	LSTM (Hochreiter & Schmidhuber) ⁶
2003	Model bahasa neural dengan representasi kata terdistribusi (Bengio dkk.) ⁴
2013	word2vec: embedding kata yang efisien (Mikolov dkk.) ⁵
2014	Sequence-to-sequence (Sutskever dkk.) ⁷ ; mekanisme attention (Bahdanau dkk.) ⁸ ; optimizer Adam ¹⁰
2015	Koneksi residual (ResNet) ⁹ ; distilasi pengetahuan (Hinton dkk.) ⁴¹
2016	BPE untuk subword (Sennrich dkk.) ¹²
2017	Transformer: "Attention Is All You Need" ¹ ; PPO ³⁴
2018	GPT-1 ¹⁴ dan BERT ¹⁵
2019	GPT-2 (1,5 miliar parameter) ¹⁶
2020	Scaling laws (Kaplan dkk.) ²³ ; GPT-3 (175 miliar parameter) ¹⁷ ; RAG ⁴⁸
2021	LoRA ⁴⁰ ; RoPE ¹³ ; makalah "Stochastic Parrots" ⁵³
2022	Chinchilla ²⁴ ; InstructGPT/RLHF ³³ ; chain-of-thought ³⁸ ; FlashAttention ³² ; Constitutional AI ³⁶ ; ChatGPT dirilis 30 November
2023	ChatGPT mencapai ±100 juta pengguna bulanan dalam dua bulan (estimasi) ⁵⁹ ; DPO ³⁵
2024	Mixtral (MoE terbuka) ²¹ ; Llama 3 (405 miliar parameter, ±15 triliun token) ¹⁸ ; FineWeb ²⁵ ; Scaling Monosemanticity ⁵⁰ ; OpenAI o1, model penalaran ³⁹ ; DeepSeek-V3 ²⁰
2025	DeepSeek-R1 (RL murni memunculkan penalaran; terbit di <i>Nature</i> September) ³⁷ ; circuit tracing Anthropic ⁵¹ ; "Why Language Models Hallucinate" ⁵²
2026	AI Index 2026: adopsi AI generatif 53% populasi; selisih kinerja AS-Tiongkok ±2,7% ⁵⁸

Glosarium

Activation (aktivasi): nilai keluaran neuron atau lapisan saat model memproses masukan. Disimpan selama pelatihan untuk dipakai backpropagation.

Adam / AdamW: optimizer standar untuk LLM. Memakai momentum dan ukuran langkah adaptif per parameter; AdamW menambahkan weight decay yang dipisahkan.

Agen (agent): sistem LLM yang menjalankan loop rencana–tindakan–pengamatan dengan memakai alat untuk menyelesaikan tugas multi-langkah.

Attention / self-attention: mekanisme yang memungkinkan setiap token menimbang dan menyerap informasi dari token-token lain berdasarkan kecocokan query dan key.

Autoregresif: menghasilkan keluaran satu token demi satu token, dengan setiap token baru menjadi bagian dari masukan langkah berikutnya.

Backpropagation: algoritma untuk menghitung gradien setiap parameter secara efisien dengan merambatkan sinyal kesalahan mundur dari keluaran ke masukan menggunakan aturan rantai.

Base model: model hasil pretraining, sebelum post-training. Pandai melanjutkan teks, tetapi belum berperilaku sebagai asisten.

Batch: sekumpulan contoh data yang diproses bersamaan dalam satu langkah pelatihan.

BPE (Byte Pair Encoding): algoritma tokenisasi yang membangun kosakata dengan menggabungkan pasangan simbol yang paling sering muncul secara berulang.

Chain-of-thought (CoT): rantai langkah penalaran yang ditulis model sebelum jawaban akhir.

Checkpoint: salinan keadaan model (parameter dan status optimizer) yang disimpan berkala selama pelatihan.

Context window (jendela konteks): jumlah maksimum token yang dapat diproses model sekaligus; memori kerja model.

Cross-entropy loss: ukuran kesalahan standar untuk model bahasa; intuisinya, seberapa "terkejut" model melihat token yang benar.

Decoder-only: varian Transformer yang hanya memakai bagian decoder dengan causal mask; dipakai hampir semua LLM generatif.

Deduplikasi: penghapusan dokumen identik atau hampir identik dari data latih.

Distilasi: melatih model kecil (murid) untuk meniru keluaran model besar (guru).

DPO (Direct Preference Optimization): metode preference tuning yang langsung mengoptimalkan model dari pasangan jawaban disukai/tidak disukai, tanpa reward model terpisah

dan tanpa loop RL.

Embedding: representasi vektor dari token; koordinat dalam "ruang makna".

Epoch: satu putaran penuh atas seluruh data latih.

Feed-forward network (FFN / MLP): bagian blok Transformer yang memproses setiap token secara terpisah; menyimpan sebagian besar parameter.

Fine-tuning: melanjutkan pelatihan model yang sudah ada pada data yang lebih spesifik.

Gradien: arah dan besar perubahan loss terhadap perubahan setiap parameter; "kemiringan tanah" dalam analogi menuruni bukit.

Gradient descent: metode optimasi yang menggeser parameter sedikit demi sedikit ke arah yang menurunkan loss.

GRPO (Group Relative Policy Optimization): algoritma RL yang dipakai DeepSeek-R1; membandingkan beberapa jawaban untuk soal yang sama dan memperkuat yang lebih baik dari rata-rata kelompok.

Halusinasi: keluaran yang terdengar meyakinkan tetapi salah atau dikarang.

Inferensi: menjalankan model yang sudah dilatih untuk menghasilkan keluaran; parameter tidak berubah.

KV cache: penyimpanan key dan value dari token-token sebelumnya agar tidak dihitung ulang saat menghasilkan token baru.

Learning rate: besar langkah pembaruan parameter dalam gradient descent.

LoRA: teknik fine-tuning hemat yang membekukan bobot asli dan hanya melatih matriks tambahan berukuran kecil.

Loss: angka yang mengukur seberapa salah prediksi model; yang ingin diminimalkan dalam pelatihan.

Mixture of Experts (MoE): arsitektur dengan banyak sub-jaringan (expert) dan router yang hanya mengaktifkan sebagian expert untuk setiap token.

Parameter: angka yang disetel selama pelatihan (bobot dan bias); tempat "pengetahuan" model tersimpan.

Post-training: semua pelatihan setelah pretraining (SFT, preference tuning, RL) yang membentuk model menjadi asisten.

PPO (Proximal Policy Optimization): algoritma reinforcement learning yang dipakai dalam RLHF klasik.

Pretraining: tahap pelatihan utama pada korpus raksasa dengan tujuan menebak token berikutnya.

Prompt: teks masukan yang diberikan ke model.

Quantization (kuantisasi): menyimpan bobot dengan presisi lebih rendah agar model lebih kecil dan cepat.

RAG (Retrieval-Augmented Generation): menyisipkan dokumen hasil pencarian ke dalam konteks sebelum model menjawab.

Reasoning model: model yang dilatih (biasanya dengan RL) untuk menghasilkan penalaran panjang sebelum menjawab.

Residual connection: jalur yang menambahkan keluaran lapisan ke masukannya, menjaga aliran informasi dan gradien di jaringan dalam.

Reward hacking: model mengakali penilai (reward model) untuk mendapat skor tinggi tanpa benar-benar menjadi lebih baik.

Reward model: model yang dilatih memprediksi seberapa besar manusia menyukai sebuah jawaban.

RLHF (Reinforcement Learning from Human Feedback): pelatihan dengan reinforcement learning memakai sinyal hadiah yang berasal dari preferensi manusia.

RLAIF / Constitutional AI: varian RLHF yang memakai penilaian AI yang dipandu prinsip tertulis sebagai pengganti sebagian penilaian manusia.

RoPE: metode pengodean posisi dengan "memutar" vektor query dan key sesuai posisinya.

Sampling / temperature / top-p: cara memilih token dari distribusi peluang; temperatur mengatur keacakan, top-p membatasi pilihan pada token-token teratas.

Scaling laws (hukum penskalaan): hubungan empiris yang dapat diprediksi antara ukuran model, data, komputasi, dan kinerja.

Self-supervised learning: pembelajaran di mana label berasal dari data itu sendiri (misalnya token berikutnya), tanpa anotasi manusia.

SFT (Supervised Fine-Tuning): fine-tuning pada contoh-contoh instruksi dan jawaban teladan.

Softmax: fungsi yang mengubah deretan skor menjadi distribusi peluang yang berjumlah 100%.

Sycophancy: kecenderungan model membenarkan atau menyanjung pengguna meskipun keliru.

Token / tokenizer: potongan teks yang menjadi unit dasar pemrosesan model / program yang memotong teks menjadi token.

Transformer: arsitektur jaringan saraf berbasis attention yang diperkenalkan pada 2017; fondasi semua LLM modern.

Kesimpulan

Setelah membedah seluruh prosesnya, beberapa pelajaran besar muncul.

Pertama, kesederhanaan tujuan, kerumitan hasil. Seluruh LLM dibangun di atas satu tugas yang nyaris remeh, yaitu menebak token berikutnya, yang dikerjakan dengan satu algoritma belajar berusia puluhan tahun, yaitu backpropagation dengan gradient descent. Yang berubah adalah skala: data dari seluruh internet, miliaran parameter, dan komputasi yang tumbuh sekitar 4–5 kali lipat per tahun²⁸. Kemampuan yang kita lihat bukan diprogram satu per satu, tetapi **tumbuh** sebagai efek samping dari upaya menebak dengan sangat baik.

Kedua, Transformer menang karena cocok dengan perangkat keras. Attention bukan hanya ide yang elegan; ia memungkinkan seluruh teks diproses paralel di GPU. Banyak kemajuan LLM adalah hasil perkawinan antara ide algoritmik dan rekayasa sistem: paralelisme, presisi campuran, FlashAttention, MoE, KV cache. Membuat LLM adalah proyek rekayasa sebesar proyek sains.

Ketiga, pengetahuan dari pretraining, perilaku dari post-training. Pretraining menghasilkan "simulator internet" yang berpengetahuan tetapi tanpa arah. Post-training, lewat SFT, umpan balik preferensi, dan RL, memilih dan menajamkan satu persona: asisten yang membantu, jujur, dan aman. Sejak 2024–2025, RL dengan hadiah terverifikasi menunjukkan bahwa model juga bisa **menemukan** strategi penalaran sendiri, bukan hanya meniru manusia³⁷.

Keempat, kelemahan LLM adalah cermin cara kerjanya. Halusinasi muncul karena model dilatih menghasilkan teks yang masuk akal dan dinilai dengan cara yang menghargai tebakan⁵². Kesulitan mengeja muncul dari tokenisasi. Sycophancy muncul dari selera manusia yang ikut terekam dalam reward model⁴³. Batas pengetahuan muncul karena parameter dibekukan setelah pelatihan. Memahami mekanisme membuat kita bisa memakai LLM dengan lebih cerdas: menyediakan konteks, meminta langkah-langkah, dan memverifikasi hal penting.

Kelima, banyak pertanyaan besar masih terbuka. Apakah LLM "memahami"? Apakah penskalaan akan terus berhasil? Apakah data akan habis? Bisakah kita benar-benar melihat ke dalam kotak hitamnya? Tidak ada jawaban final hingga September 2026. Yang pasti, LLM bukan sihir dan bukan sekadar autocomplete. Ia adalah kompresi statistik yang sangat besar atas bahasa manusia, yang dilatih untuk berperilaku sebagai asisten, dengan kekuatan dan kelemahan yang bisa dijelaskan, dan karena itu bisa dipelajari, diperbaiki, dan diawasi.

Bacaan Lanjutan

Untuk memulai (tanpa matematika)

- **Andrej Karpathy, "Deep Dive into LLMs like ChatGPT" (video YouTube, 2025):** kuliah 3,5 jam yang membahas seluruh pipeline dari data sampai RL untuk penonton umum. Paling dekat dengan cakupan dokumen ini⁶⁰.
- **3Blue1Brown, seri "Neural Networks" (YouTube):** animasi visual terbaik tentang jaringan saraf, gradient descent, backpropagation, dan attention.
- **Jay Alammar, "The Illustrated Transformer" (blog):** penjelasan bergambar tentang Transformer yang menjadi rujukan klasik.

Untuk yang ingin lebih teknis

- **Andrej Karpathy, "Neural Networks: Zero to Hero" dan nanoGPT:** membangun jaringan saraf, backpropagation, dan GPT kecil dari nol dengan kode.
- **Vaswani dkk., "Attention Is All You Need" (2017):** makalah asli Transformer, singkat dan relatif mudah dibaca¹.
- **Llama Team, "The Llama 3 Herd of Models" (2024):** laporan teknis paling lengkap dan terbuka tentang cara membuat LLM skala terdepan, dari data sampai post-training¹⁸.
- **DeepSeek-AI, "DeepSeek-R1" (Nature, 2025):** pintu masuk terbaik untuk memahami RL dan model penalaran³⁷.
- **Ouyang dkk., "InstructGPT" (2022):** fondasi RLHF dan asal-usul ChatGPT³³.
- **Hoffmann dkk., "Chinchilla" (2022):** memahami hukum penskalaan dan keseimbangan model-data²⁴.

Untuk perspektif kritis dan interpretabilitas

- **Anthropic, "Tracing the thoughts of a large language model" (2025) dan "Scaling Monosemanticity" (2024):** apa yang terjadi di dalam model, dengan visualisasi yang bisa diikuti⁵¹⁵⁰.
- **Bender, Gebru dkk., "Stochastic Parrots" (2021):** kritik paling berpengaruh terhadap LLM⁵³.
- **Stanford HAI, AI Index Report (terbit tahunan):** data terbaik tentang tren biaya, kemampuan, adopsi, dan kebijakan AI⁵⁷⁵⁸.
- **Buku: Simon J.D. Prince, *Understanding Deep Learning* (MIT Press, 2023, tersedia gratis daring):** buku teks modern yang jelas bagi yang siap dengan sedikit matematika.

Catatan Kaki

1. Vaswani, A. dkk., *Attention Is All You Need*, NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
2. Rumelhart, D. E., Hinton, G. E. & Williams, R. J., *Learning representations by back-propagating errors*, Nature 323, 1986. <https://www.nature.com/articles/323533a0>
3. Schmidhuber, J., *Who Invented Backpropagation?*, IDSIA. <https://people.idsia.ch/~juergen/who-invented-backpropagation.html>
4. Bengio, Y. dkk., *A Neural Probabilistic Language Model*, JMLR 3, 2003. <https://www.jmlr.org/papers/v3/bengio03a.html>
5. Mikolov, T. dkk., *Efficient Estimation of Word Representations in Vector Space*, 2013. <https://arxiv.org/abs/1301.3781>
6. Hochreiter, S. & Schmidhuber, J., *Long Short-Term Memory*, Neural Computation, 1997. <https://www.bioinf.jku.at/publications/older/2604.pdf>
7. Sutskever, I., Vinyals, O. & Le, Q. V., *Sequence to Sequence Learning with Neural Networks*, 2014. <https://arxiv.org/abs/1409.3215>
8. Bahdanau, D., Cho, K. & Bengio, Y., *Neural Machine Translation by Jointly Learning to Align and Translate*, 2014. <https://arxiv.org/abs/1409.0473>
9. He, K. dkk., *Deep Residual Learning for Image Recognition*, 2015. <https://arxiv.org/abs/1512.03385>
10. Kingma, D. P. & Ba, J., *Adam: A Method for Stochastic Optimization*, 2014. <https://arxiv.org/abs/1412.6980>
11. Loshchilov, I. & Hutter, F., *Decoupled Weight Decay Regularization*, 2017. <https://arxiv.org/abs/1711.05101>
12. Sennrich, R., Haddow, B. & Birch, A., *Neural Machine Translation of Rare Words with Subword Units*, ACL, 2016. <https://aclanthology.org/P16-1162/>
13. Su, J. dkk., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, 2021. <https://arxiv.org/abs/2104.09864>
14. Radford, A. dkk., *Improving Language Understanding by Generative Pre-Training*, OpenAI, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
15. Devlin, J. dkk., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2018. <https://arxiv.org/abs/1810.04805>
16. Radford, A. dkk., *Language Models are Unsupervised Multitask Learners*, OpenAI, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
17. Brown, T. dkk., *Language Models are Few-Shot Learners*, NeurIPS, 2020. <https://arxiv.org/abs/2005.14165>
18. Llama Team, AI @ Meta, *The Llama 3 Herd of Models*, 2024. <https://arxiv.org/abs/2407.21783>
19. Meta, *Llama 3.1 Model Card*, 2024. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md
20. DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024. <https://arxiv.org/abs/2412.19437>
21. Jiang, A. Q. dkk., *Mixtral of Experts*, 2024. <https://arxiv.org/abs/2401.04088>
22. Ainslie, J. dkk., *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, 2023. <https://arxiv.org/abs/2305.13245>
23. Kaplan, J. dkk., *Scaling Laws for Neural Language Models*, 2020. <https://arxiv.org/abs/2001.08361>
24. Hoffmann, J. dkk., *Training Compute-Optimal Large Language Models*, 2022. <https://arxiv.org/abs/2203.15556>
25. Penedo, G. dkk. (Hugging Face), *FineWeb: decanting the web for the finest text data at scale*, 2024. <https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1>

26. Villalobos, P. dkk., *Will we run out of data? Limits of LLM scaling based on human-generated data*, ICML, 2024. <https://arxiv.org/abs/2211.04325>
27. Shumailov, I. dkk., *AI models collapse when trained on recursively generated data*, Nature 631, 2024. <https://www.nature.com/articles/s41586-024-07566-y>
28. Epoch AI, *Training compute of frontier AI models grows by 4-5x per year*, 2024. <https://epoch.ai/blog/training-compute-of-frontier-ai-models-grows-by-4-5x-per-year>
29. Narayanan, D. dkk., *Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM*, SC, 2021. https://people.eecs.berkeley.edu/~matei/papers/2021/sc_megatron_lm.pdf
30. Rajbhandari, S. dkk., *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*, 2019. <https://arxiv.org/abs/1910.02054>
31. Micikevicius, P. dkk., *Mixed Precision Training*, 2017. <https://arxiv.org/abs/1710.03740>
32. Dao, T. dkk., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, NeurIPS, 2022. <https://arxiv.org/abs/2205.14135>
33. Ouyang, L. dkk., *Training language models to follow instructions with human feedback*, NeurIPS, 2022. <https://arxiv.org/abs/2203.02155>
34. Schulman, J. dkk., *Proximal Policy Optimization Algorithms*, 2017. <https://arxiv.org/abs/1707.06347>
35. Rafailov, R. dkk., *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*, NeurIPS, 2023. <https://arxiv.org/abs/2305.18290>
36. Bai, Y. dkk. (Anthropic), *Constitutional AI: Harmlessness from AI Feedback*, 2022. <https://arxiv.org/abs/2212.08073>
37. DeepSeek-AI, *DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning*, Nature 645, 2025. <https://www.nature.com/articles/s41586-025-09422-z>
38. Wei, J. dkk., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, 2022. <https://arxiv.org/abs/2201.11903>
39. OpenAI, *Learning to Reason with LLMs*, 2024. <https://openai.com/index/learning-to-reason-with-llms/>
40. Hu, E. J. dkk., *LoRA: Low-Rank Adaptation of Large Language Models*, 2021. <https://arxiv.org/abs/2106.09685>
41. Hinton, G., Vinyals, O. & Dean, J., *Distilling the Knowledge in a Neural Network*, 2015. <https://arxiv.org/abs/1503.02531>
42. Gao, L., Schulman, J. & Hilton, J., *Scaling Laws for Reward Model Overoptimization*, 2022. <https://arxiv.org/abs/2210.10760>
43. Sharma, M. dkk. (Anthropic), *Towards Understanding Sycophancy in Language Models*, 2023. <https://arxiv.org/abs/2310.13548>
44. Schick, T. dkk., *Toolformer: Language Models Can Teach Themselves to Use Tools*, 2023. <https://arxiv.org/abs/2302.04761>
45. Holtzman, A. dkk., *The Curious Case of Neural Text Degeneration*, 2019. <https://arxiv.org/abs/1904.09751>
46. Kwon, W. dkk., *Efficient Memory Management for Large Language Model Serving with PagedAttention*, 2023. <https://arxiv.org/abs/2309.06180>
47. Leviathan, Y., Kalman, M. & Matias, Y., *Fast Inference from Transformers via Speculative Decoding*, 2022. <https://arxiv.org/abs/2211.17192>
48. Lewis, P. dkk., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, 2020. <https://arxiv.org/abs/2005.11401>
49. Liu, N. F. dkk., *Lost in the Middle: How Language Models Use Long Contexts*, 2023. <https://arxiv.org/abs/2307.03172>
50. Templeton, A. dkk. (Anthropic), *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>

51. Anthropic, *Tracing the thoughts of a large language model*, 2025. <https://www.anthropic.com/research/tracing-thoughts-language-model>
52. Kalai, A. T. dkk. (OpenAI), *Why Language Models Hallucinate*, 2025. <https://arxiv.org/abs/2509.04664>
53. Bender, E. M., Gebru, T. dkk., *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?*, FAccT, 2021. <https://dl.acm.org/doi/10.1145/3442188.3445922>
54. Li, K. dkk., *Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task*, 2022. <https://arxiv.org/abs/2210.13382>
55. Wei, J. dkk., *Emergent Abilities of Large Language Models*, 2022. <https://arxiv.org/abs/2206.07682>
56. Schaeffer, R., Miranda, B. & Koyejo, S., *Are Emergent Abilities of Large Language Models a Mirage?*, 2023. <https://arxiv.org/abs/2304.15004>
57. Stanford HAI, *Artificial Intelligence Index Report 2025, Chapter 1: Research and Development*, 2025. https://hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf
58. Stanford HAI, *Inside the AI Index: 12 Takeaways from the 2026 Report*, 2026. <https://hai.stanford.edu/news/inside-the-ai-index-12-takeaways-from-the-2026-report>
59. Reuters (via Yahoo Finance), *ChatGPT sets record for fastest-growing user base - analyst note*, Februari 2023. <https://finance.yahoo.com/news/chatgpt-sets-record-fastest-growing-190911828.html>
60. Karpathy, A., *Deep Dive into LLMs like ChatGPT* (video, 2025), ringkasan oleh L. Wu. <https://lawwu.github.io/posts/2025-02-24-karpathy-deep-dive-llms/index.html>